How Coding Agents Work

An Architecture Handbook — distilled from five production agents

claude-code · opencode · openclaude · codex · goose
reverse-engineered from source, with file:line citations throughout

Contents

- Introduction

Part I — The Core

- 1. The Agent Loop
- 2. Messages, Blocks & Conversation State
- 3. The Tool System
- 4. The Model & Provider Layer
- 5. Prompt Construction & Caching
- 6. Context Management & Compaction

Part II — Control & Safety

- 7. Permissions, Approval & Sandboxing
- 8. Hooks
- 9. Memory & Context Files

Part III — Extensibility

- 10. Skills
- 11. Slash & Custom Commands
- 12. Output Styles
- 13. Subagents & Orchestration
- 14. MCP (Model Context Protocol)

Part IV — Runtime

- 15. Sessions, Persistence, Todos & Background Tasks

Part V — Building It

- 16. End-to-End Scenarios
- 17. The Lean Build

Introduction

This handbook explains how a modern coding agent actually works — not the marketing view, the wiring. It is reverse-engineered from five real codebases and written for one purpose: so you can build your own lean agent framework and know exactly which parts are load-bearing and which are product decoration.

The mental model

An agent is a **loop around a language model that can call tools**. The model never touches your machine directly; it can only *propose* actions as structured `tool_use` requests. The harness executes those actions, feeds the results back as the next message, and asks the model again. This repeats until the model stops requesting tools. That is the whole idea:

```
messages = [ user(task) ]
loop:
   reply   = model(system_prompt, tools, messages)   # the model proposes
   messages.append(assistant(reply))
   calls   = [ b for b in reply.content if b.type == "tool_use" ]
   if calls is empty:            # <- the model asked for nothing => it is done
       return reply
   results = [ run_with_permission(c) for c in calls ]   # the harness acts
   messages.append(user(results))                        # feed results back
```

Everything else in this book — streaming, context compaction, skills, hooks, subagents, MCP, sandboxes, sessions — is an elaboration of those ten lines.

The word for it: the "harness"

That surrounding loop-plus-tools-plus-context machinery has a name that is suddenly everywhere: the **agent harness**. The framing worth internalizing is a small equation — **Agent = Model + Harness**. The model is the interchangeable brain; the harness is everything in this book that turns it into something that acts. The leaked Claude Code source described itself as exactly that: *"the skeleton not the brain."*

The term is trending because of a backlash toward **minimal** harnesses. Once you separate the two halves, you notice most frameworks over-scaffold — giant system prompts and dozens of tools that actually degrade the model by crowding its context. The current exemplar, **pi.dev**, strips the harness to four tools (`read/write/edit/bash`) and a system prompt under 1,000 tokens, then lets you *extend it by writing code the agent loads into itself* (Ch. 3, 5, 17). Keep that lens while reading: for every component here, ask "is this the kernel, or is it weight?"

The convergence thesis

Five independently built agents — two of them in Rust, three in TypeScript — converge on the **same kernel**: a `while(true)` turn loop, a `{name, JSON-schema, call}` tool abstraction, a streaming provider call with tool-call extraction, threshold-triggered context compaction, and an allow/ask/deny permission gate. The other ~95% of each codebase is surface: UIs, telemetry, provider breadth, enterprise/cloud, and multi-agent choreography. If you build the kernel well, you have a real agent. This book teaches the kernel first (Part I), then the genuinely-useful elaborations (Parts II–IV), then how to assemble a lean one (Part V).

The five sources

Repo	Lang	Studied for
`claude-code`	TS (leaked)	the canonical architecture; battle-tested loop ergonomics
`opencode`	TS	cleanest modern structure; persistence-as-source-of-truth
`openclaude`	TS	multi-provider via a canonical-format shim
`codex`	Rust	strongest safety: real OS sandbox; typed protocol
`goose`	Rust	MCP as the universal tool bus; dual message visibility

Seven laws that recur in every chapter

1. **The exit signal is the absence of a `tool_use` block — not the provider's `stop_reason`.** Four of five repos explicitly distrust `stop_reason`.
2. **Everything is a tool result.** Validation failures, permission denials, and tool exceptions are all fed back to the model as `is_error` results, never thrown. The loop has one happy path.
3. **The pairing invariant.** Every emitted `tool_use` must get a matching `tool_result` before the next model call, or the API rejects the request.
4. **Keep the system-prompt prefix byte-stable** so the provider's prompt cache hits; push dynamic facts to the end or into a user message.
5. **Compact at a threshold, don't delete.** Summarize near ~80% of the window and rebuild history; mark old turns invisible rather than dropping them.
6. **Gate before side effects.** The permission decision happens before a tool runs, not after.
7. **A subagent is just the loop, recursed** with a child context and a filtered tool set. There is no separate "agent runtime".

How to read this

Chapters are self-contained. Each follows the same shape: *what & why → how it works (with a flow and pseudocode) → in the wild (how the five repos do it, with `repo/path:line` citations) → a concrete scenario → a lean take.* Citations point into `repos/<name>/...`; verify any of them by opening the file. Pseudocode is language-agnostic; real excerpts are labelled by language.

> The `claude-code` repository is a leaked snapshot of Anthropic's proprietary code. It is cited here for **understanding patterns only** — re-implement the ideas, do not copy its source into your own framework.

1. The Agent Loop

The agent loop is the engine: call the model, run any tools it asked for, feed the results back, repeat until it stops asking. Everything else in this handbook — tools, permissions, compaction, sub-agents — hangs off this loop. Get its exit condition and its tool-pairing invariant right and the rest is detail.

How it works

A turn is one user request driven to completion through any number of model calls. Each model call streams back an assistant message that may contain `text` blocks and `tool_use` blocks. If it contains tool calls, the loop must execute them, append their results, and call the model again — the model has not finished. If it contains no tool calls, the model is done and the turn ends.

The decisive design choice, shared by four of the five repos, is **what signals "done."** The naive answer is the provider's `stop_reason` (`end_turn` vs `tool_use`). Every mature loop distrusts it: providers set it inconsistently, and some return `stop`/`end_turn` even when the assistant content contains tool calls. The authoritative signal is **the absence of a `tool_use` block in the assistant's content** (claude-code `src/query.ts:554`). You scan the content you actually received; you do not ask the provider's metadata how it went.

Three invariants make the loop robust:

- **Tool-use ↔ tool-result pairing.** The Messages API rejects a request where an assistant `tool_use` block has no matching `tool_result` in the next user turn. So *every* emitted `tool_use` must get a result before the next call — including on abort, error, or model-fallback paths, where you back-fill a synthetic error result (claude-code `yieldMissingToolResultBlocks`, `src/query.ts:123`).
- **Errors are tool results, not exceptions.** Schema-validation failures, permission denials, and tool exceptions all become `is_error` tool_result blocks fed back to the model so it self-corrects. The loop has one happy path and no exception-driven control flow.
- **Parse, then execute.** Tool calls are collected from the completed stream, then executed — read-only/concurrency-safe ones in parallel, writers serially.

Flow

1. (Optional) compact if the estimated token count is over threshold.
2. Assemble system prompt + message history + tool schemas; call the model (streaming).
3. Accumulate streamed content blocks by index; assemble each `tool_use` input from JSON deltas, parsing only at block close. Append the assistant message.
4. **Exit test:** if the assistant content has no `tool_use` block, return — the turn is complete. Do not consult `stop_reason`.
5. Otherwise, for each tool call: validate input, run the permission gate, execute. Any failure becomes an `is_error` tool_result. Run safe tools in parallel, writers serially.
6. Append a user message carrying all the tool_result blocks (one per tool_use). Check turn/budget guards; `continue`.

```
async function runTurn(messages, tools, provider, gate, ctx):
   while true:
       if estimateTokens(messages) > window - BUFFER:
           messages = compact(messages, provider)          # one summary pass

       assistant  = emptyAssistant()
       toolCalls  = []
       for await ev in provider.stream(system, messages, toolSchemas):
           case "text-delta":  assistant.appendText(ev.text)
```

```
            case "tool-call":   toolCalls.push(ev)           # input assembled at close
                                assistant.appendToolUse(ev)
            case "finish":      break                        # IGNORE ev.stop_reason
        messages.push(assistant)

        if toolCalls.isEmpty():                              # THE exit signal
            return assistant                                 # no tool_use ⇒ turn done

        results = []
        for batch in partitionBySafety(toolCalls, tools):    # safe ⇒ parallel batch
            results += await parallelMap(batch, tc => runOne(tc, tools, gate, ctx))
        # invariant: results has exactly one entry per toolCalls entry
        messages.push(userMessage(results))
        if ++turn > MAX_TURNS: return                        # budget guard
        continue

function runOne(tc, tools, gate, ctx):
   tool   = tools.byName(tc.name)
   parsed = tool.schema.safeParse(tc.input)
   if not parsed.ok:        return toolResult(tc.id, parsed.error, isError=true)
   decision = gate.decide(tool, parsed.value, ctx)
   if decision == "deny":   return toolResult(tc.id, "denied", isError=true)
   if decision == "ask" and not ctx.askUser(...):
                            return toolResult(tc.id, "rejected", isError=true)
   try:    return toolResult(tc.id, tool.call(parsed.value, ctx))
   except e: return toolResult(tc.id, e.message, isError=true)  # error ⇒ result
```

In the wild

Repo	Loop shape	Exit signal
claude-code	`while(true)` async-gen, mutable `State`+`continue`	no `tool_use` block: `needsFollowUp` false `query.ts:1062`
openclaude	same async-gen, single reassigned `state`	streamed `tool_use` absent; `stop_reason` distrusted
opencode	`while(true)`, re-derives state from SQLite each turn	last finish ≠ `tool-calls` AND no tool parts `prompt.ts:1111`
codex	outer steer loop + inner `run_turn`	`needs_follow_up` false `turn.rs:318`
goose	`async_stream loop{}` of typed `AgentEvent`	`no_tools_called` (then overrides) `agent.rs:2513`

Divergences worth knowing:

- **Where loop state lives.** claude-code/openclaude carry one mutable `State` and express every retry path (model fallback, compaction, budget, recovery) as `state = {...}; continue` (`query.ts:204` region), so the loop is flat and every branch is uniform. opencode holds **no** mutable conversation object: each turn it reloads history from SQLite and re-derives "latest user / latest finished assistant / pending tasks" by max id (`prompt.ts:1096`), making cancel/resume correct by construction. codex resends the full transcript every turn.
- **The exit predicate, precisely.** claude-code sets `needsFollowUp = true` the moment a `tool_use` block arrives during streaming and exits when it is still false afterward (`query.ts:558`, `:834`, `:1062`). codex folds three conditions into one boolean: a tool call was seen, OR the provider reported `end_turn:false` (`turn.rs:2233`), OR there is queued user input (`turn.rs:318`). goose starts `no_tools_called = true` (`agent.rs:2002`), flips it false when a tool runs (`agent.rs:2366`), then *vetoes* the stop with goal/grind/stop-hook nudges (`agent.rs:2536`, `:2555`) — the one repo that complicates the clean rule.

- **The pairing back-fill.** Only claude-code/openclaude make it explicit: `yieldMissingToolResultBlocks` walks the assistant messages and emits a synthetic `is_error` tool_result for any `tool_use` left unpaired by an abort, image error, or model swap (`query.ts:123`, called at `:900`, `:984`, `:1025`). The others get pairing for free because they only append results in lockstep with calls.
- **Parallel vs serial execution.** claude-code partitions consecutive concurrency-safe tools into parallel batches (cap 10, `CLAUDE_CODE_MAX_TOOL_USE_ CONCURRENCY`) and runs everything else serially (`toolOrchestration.ts:26`, `:91`). codex enforces the same policy with a single `RwLock<()>`: parallel-safe tools take a read lock, writers take the write lock, so reads overlap and writes are exclusive (`tools/parallel.rs:117`). goose drains many `ToolStream`s in a `tokio::select!`. opencode delegates execution to the Vercel AI SDK and does no explicit batching.
- **Guards.** claude-code stops with `reason: 'max_turns'` when the turn count exceeds `maxTurns` (`query.ts:1705`) and has a hard `blocking_limit` token stop (`query.ts:646`). codex bounds turns with cancellation tokens threaded through `run_turn` rather than a counter (`turn.rs:148`).

Scenario

User: "Rename `getUser` to `fetchUser` across the repo." One turn, two model calls.

1. **Call 1.** The loop sends history + tool schemas. The model streams two `tool_use` blocks: `grep {pattern:"getUser"}` and `read {path:"a.ts"}`. Each block's `input` is assembled from `input_json_delta` fragments and JSON-parsed at `content_block_stop`. The `message_delta` carries `stop_reason` — ignored. The assistant message is appended; `needsFollowUp` is true because two `tool_use` blocks exist.
2. **Execute.** Both tools are read-only, so they form one parallel batch and run concurrently. `grep` returns three files; `read` returns the file. Two `tool_result` blocks, paired by `tool_use_id`, go into one user message. `continue`.
3. **Call 2.** The model now streams one `edit` `tool_use`. `edit` is a writer, so it runs in its own serial batch. The permission gate returns `ask`; the user approves. The edit succeeds; its `tool_result` is appended.
4. **Call 3.** The model streams a single `text` block: "Renamed across 3 files." No `tool_use` block. The exit test fires: `toolCalls.isEmpty()` → return. Turn complete after three model calls and three tools.

Counter-case showing the invariant: had the user aborted during the `edit`, the loop would back-fill a synthetic `is_error` tool_result for the in-flight `edit` `tool_use` (`query.ts:123`) so the persisted history stays paired and the next request does not 400.

Lean take

- **One flat `while(true)` with `continue`.** Hold a single mutable `{messages, turn, transition}` and reassign it per iteration. Every retry path (compaction, fallback, backoff) becomes another `continue` branch — no nested state machine (claude-code `query.ts:204`).
- **Exit on absence of a `tool_use` block. Never trust `stop_reason`.** This is the single most important line in your loop (claude-code `query.ts:554`).
- **Enforce pairing.** After collecting tool calls, guarantee exactly one `tool_result` per `tool_use` before the next call; on any abort/error path, back-fill a synthetic `is_error` result (claude-code `query.ts:123`).
- **Make every failure a result.** Validation, denial, and exceptions return `is_error` tool_result blocks; the loop never throws to control flow.
- **Parallelize by one `isReadOnly` predicate, serialize writers.** A free latency win with no scheduler (claude-code `toolOrchestration.ts:91`; codex `parallel.rs:117`). Skip it in v0 if you want — correctness does not depend on it.

- **Skip:** goal/grind/stop-hook "keep going" overrides (goose `agent.rs:2536`) — they fight the clean stop rule and add nondeterminism. Add a `MAX_TURNS` guard and stop there.

2. Messages, Blocks & Conversation State

The conversation is the agent's entire working memory: an ordered list of role-tagged messages, each carrying a list of typed content blocks. This chapter fixes the wire format every loop reads and writes, decides where that list lives between turns, and states the one structural invariant that keeps the provider from rejecting your next request.

How it works

Every provider that supports tool use converges on the same two-level shape: a **message** with a `role` and a `content` array, and **content blocks** that are a tagged union — `text`, `tool_use` (the model asks to call a tool), `tool_result` (you feed the output back), and `thinking`/`reasoning`. A turn is a sequence of these messages; the loop reads the latest assistant message's blocks to decide whether to continue (see Chapter 1) and appends new messages as it runs tools.

Two design decisions sit on top of this format. First, **where the list lives**. The mutable in-memory camp (claude-code, openclaude, codex) holds one growing array and mutates it each turn; the reload camp (opencode) persists every message to a store and re-derives "latest user / latest finished assistant / pending work" from it on every turn, holding no mutable conversation object at all. Second, **dual visibility**: goose tags each message with `user_visible` and `agent_visible` flags so compaction can hide a message from the model while keeping it in the user's transcript — non-destructive by construction.

The invariant that binds it all: **every `tool_use` block must have a matching `tool_result`** (paired by id) before the next model call, or the API 400s. This is enforced at the data layer, not the loop layer.

Flow

1. User input becomes a `user` message with one or more content blocks (text, images); append to history.
2. Build the request: system prompt + tool schemas + the message list (full, or sliced after the last compaction boundary).
3. Stream the assistant reply; accumulate blocks by index into one `assistant` message: `text`, `thinking`, and `tool_use` (args assembled from JSON deltas).
4. For each `tool_use`, run the tool, wrap the output as a `tool_result` block carrying the same `id`, and append it inside a `user` message.
5. Before the next request, **normalize**: ensure every emitted `tool_use` has a `tool_result`; back-fill a synthetic error result for any unpaired one; drop orphan results whose call was compacted away.
6. Loop. State either persists in the mutable array or is re-derived from the store.

```
Message  = { role: 'user'|'assistant', content: ContentBlock[] }
ContentBlock =
 | { type:'text',        text }
 | { type:'thinking',    thinking, signature }
 | { type:'tool_use',    id, name, input }          # assistant -> "run this"
 | { type:'tool_result', tool_use_id, content, is_error? }  # user -> "here"

# the pairing invariant, run at request-build time:
normalize(history):
 emitted  = { b.id          for msg.b in history if b.type=='tool_use' }
 resolved = { b.tool_use_id for msg.b in history if b.type=='tool_result' }
 for id in emitted - resolved:        # tool_use with no result
     append user msg [tool_result(id, "aborted/error", is_error=true)]
 for r in history if r.tool_use_id not in emitted:   # orphan result
     drop r
 assert no duplicate tool_use ids     # API: "tool_use ids must be unique"
```

In the wild

Repo	Block union lives in	State model
claude-code	inferred `Message` (not in tree)	mutable `Message[]`
openclaude	`types/message.ts:372`	mutable `Message[]`
codex	`ResponseItem` enum `models.rs:934`	mutable `Vec<ResponseItem>`
goose	`MessageContent` `message.rs:271`	`Vec<Message>` + flags
opencode	AI-SDK parts	reload from SQLite

- **openclaude** is the cleanest reference for the canonical Anthropic shape: `UserMessage` (`types/message.ts:90`) and `AssistantMessageContent` (`:144`, `{role,content,id,model,usage}`) wrap `BetaContentBlock` / `ContentBlockParam` from the SDK (`:29`). A *normalized* message holds exactly one block per message (`NormalizedUserMessage`, `:420`) so the loop can reason block-by-block.
- **claude-code** carries the live array in `QueryEngine.mutableMessages` (`QueryEngine.ts:186,202`) and a per-turn `State.messages` (`query.ts:205`). It distrusts `stop_reason:'tool_use'` and keys the loop on block presence (`query.ts:554`).
- **codex** models messages as `ResponseItem::Message{role,content:Vec<ContentItem>}` (`models.rs:943`) where `ContentItem` is `InputText|InputImage|OutputText` (`models.rs:844`); tool calls are *sibling* items (`FunctionCall`/`FunctionCallOutput`, `models.rs:996,1033`), not blocks inside a message. History is a `Vec<ResponseItem>` (`history.rs:38-40`) resent in full via `for_prompt` (`history.rs:141`).
- **goose** puts visibility on `MessageMetadata` (`message.rs:672`): `user_visible` (`:674`) and `agent_visible` (`:676`), both defaulting `true` (`:689`). `agent_only()` (`:699`) and `with_agent_invisible()` (`:726`) flip them; token counting and provider serialization filter on `is_agent_visible()` (`token_counter.rs:143`, `message.rs:1080`).
- **opencode** holds *no* mutable conversation: `MessageV2.latest(msgs)` (`message-v2.ts:585`) re-derives latest-user / latest-assistant / latest-finished / pending tasks by max id each turn, and `prompt.ts:1096` drives stop/continue from that. `filterCompacted` (`message-v2.ts:521`) reorders for the model without mutating storage.

Divergence to note: claude-code/openclaude/goose nest tool blocks *inside* messages (Anthropic shape); codex makes calls and outputs *peer items* in a flat vec (Responses shape). The pairing invariant is identical either way — it is about `id` ↔ `tool_use_id`/`call_id` matching, not nesting.

Scenario

The model asks to run two tools but the process is killed after the first finishes. History at that moment (Anthropic shape):

```
user:      [text "rename foo.py and run tests"]
assistant: [tool_use id=A name=edit_file ..., tool_use id=B name=bash ...]
user:      [tool_result tool_use_id=A "ok"]        # B never ran
```

On the next turn the loop builds the request and runs the normalizer. It sees `emitted={A,B}` but `resolved={A}`, so B is unpaired. Rather than abort or 400, it back-fills a synthetic result (claude-code `yieldMissingToolResultBlocks`, `query.ts:123-148`; openclaude `ensureToolResultPairing`, `messages.ts:6010`; codex `ensure_call_outputs_present`, `normalize.rs:17`, emitting `"aborted"`):

```
user: [tool_result tool_use_id=B content="<interrupted>" is_error=true]
```

Now `emitted == resolved`, the request is well-formed, and the model sees that B was interrupted and can retry it. The inverse case — a `tool_result` whose `tool_use` was dropped by compaction — is handled by the orphan sweep (codex `remove_orphan_outputs`, `normalize.rs:144`; openclaude strips the orphan result, `messages.ts:6044`). Both run inside `normalize_history` at prompt-build time (`history.rs:359-364`), so the loop body never has to think about pairing.

Lean take

- **Adopt the Anthropic block shape verbatim:** `{role, content[]}` with `text | tool_use | tool_result | thinking`. It is the most widely supported tagged union and maps cleanly onto OpenAI's flat items if you ever port. Don't invent your own.
- **Pair by id, enforce once, at request-build time.** Write one `normalize()` that back-fills missing `tool_result`s (with `is_error:true`), drops orphans, and de-dups `tool_use` ids. This single function removes a whole class of 400s and lets the loop stay dumb.
- **Start with one mutable in-memory array.** Reload-from-store (opencode) is genuinely cleaner for cancel/resume/compaction correctness, but it needs a persistence layer and monotonic ids first. Add it when you need durable sessions, not before.
- **Steal goose's dual visibility flags early** (`user_visible`/`agent_visible`). They are two booleans on each message but make compaction non-destructive: hide old turns from the model, keep them in the transcript. Filter on `agent_visible` wherever you serialize to the provider or count tokens.
- **Keep `thinking` blocks attached to their turn.** They must travel with the assistant message (and survive across the following `tool_result`) or the provider rejects the trajectory — treat them as load-bearing, not cosmetic.
- **Skip** the dozen system/UI message subtypes (boundaries, summaries, metrics) the production repos carry; those are transcript-rendering and telemetry, not conversation state.

3. The Tool System

The tool system is the contract between the model and the world: a list of named, schema-described actions the model can request, plus the machinery to validate, gate, run, and shape their results back into the conversation. It is the only part of the loop that touches the real filesystem, shell, and network.

How it works

A tool is a small record: a `name`, a model-facing `description`, an `inputSchema`, and a `call(input, ctx)` implementation. Two extra fields carry most of the system's intelligence: `isReadOnly` (drives both parallelism and auto-approval) and an optional `checkPermissions(input)` for tool-specific risk logic (e.g. bash sub-command rules). Everything else a real repo piles on — UI renderers, telemetry, classifier inputs — is product surface, not mechanism.

Three invariants make the system robust. First, the schema lives **next to** the executing code so the model-facing description and the validator cannot drift. Second, the schema is rendered to JSON Schema and handed to the provider as the `tools` array; the model sees *only* `name`, `description`, and that schema. Third, **every failure becomes a tool result, not an exception** — a schema-validation failure, a permission denial, and a thrown error all turn into an `is_error` tool_result fed back to the model so it self-corrects. The loop has one happy path.

The registry is deliberately not a framework: it is a hand-written array of built-ins plus a merge step that folds in external (MCP/plugin) tools. Built-ins win name conflicts, and the merged list is sorted for prompt-cache stability.

Flow

1. **Declare**: each tool is built through a factory (`buildTool`) that fills fail-closed defaults — `isReadOnly→false`, `checkPermissions→allow` (defer to the global gate) — so a new tool is safe-by-default and tiny to write.
2. **Register**: collect built-ins into a flat array; merge MCP/plugin tools; sort by name; built-ins win conflicts.
3. **Render**: for each tool, `description = tool.prompt(...)` and `input_schema = zodToJsonSchema(tool.inputSchema)` (or a raw JSON Schema for MCP tools). Cache the serialized result per session for prompt-cache stability.
4. **Validate**: on a tool call, `inputSchema.safeParse(input)`; on failure, return a `<tool_use_error>` result — no crash.
5. **Gate**: run `checkPermissions` + the global allow/ask/deny gate before any side effect; deny returns an `is_error` result.
6. **Run**: `await tool.call(parsed, ctx)`; catch exceptions into error results.
7. **Shape**: map the output to a tool_result block; if it exceeds `maxResultSizeChars`, spill to disk and replace the content with a preview + pointer.

```
buildTool(def):
   return { ...TOOL_DEFAULTS, ...def }   # defaults: isReadOnly=false, perms=allow

renderForModel(tools):                    # what the API sees
   return tools.map(t => {
       name:         t.name,
       description:  await t.prompt(ctx), # method, can vary by mode/agent
       input_schema: t.inputJSONSchema ?? zodToJsonSchema(t.inputSchema),
   })

executeOne(tc, tools, gate, ctx):
   tool   = tools.byName(tc.name)        # unknown name => is_error result
   parsed = tool.inputSchema.safeParse(tc.input)
   if not parsed.ok:
       return result(tc.id, "<tool_use_error>"+parsed.error+"</tool_use_error>",
                     isError=true)        # validation failure IS the result
```

```
if (await gate.decide(tool, parsed.value, ctx)) == deny:
    return result(tc.id, "Permission denied.", isError=true)
try:    out = await tool.call(parsed.value, ctx)
except e: return result(tc.id, e.message, isError=true)
block = tool.mapResultToBlock(out, tc.id)
if size(block) > tool.maxResultSizeChars:
    block = spillToDisk(block)         # preview + file pointer
return block
```

In the wild

Concern	claude-code / openclaude	codex	goose
Tool shape	structural `Tool<In,Out>`	one `ToolExecutor` trait	`rmcp::Tool` + annotations
Spec+run	separate object fields	`spec()`+`handle()`	served via MCP
Schema source	`zodToJsonSchema`	serde enum `ToolSpec`	`schemars` from struct
Defaults	`buildTool` fail-closed	trait defaults	`ToolAnnotations`

- **claude-code**: the interface is `Tool<Input,Output>` with the core members `name`, `inputSchema` (Zod), `prompt()` (the model-facing description), `call()`, `checkPermissions()`, and `mapToolResultToToolResultBlockParam()` (`claude-code/src/Tool.ts:394,500,518,557`). `buildTool` spreads `TOOL_DEFAULTS` with fail-closed values — `isReadOnly→false`, `isConcurrencySafe→false`, `checkPermissions→allow` (`claude-code/src/Tool.ts:757,783`). Schema rendering is `zodToJsonSchema(tool.inputSchema)` (or `tool.inputJSONSchema` for MCP) inside a per-session cache (`claude-code/src/utils/api.ts:160,171`).
- **openclaude** is the same TS interface in a multi-provider fork: `buildTool` at `openclaude/src/Tool.ts:812`, `TOOL_DEFAULTS` at `:786`. Because the canonical wire format is Anthropic, MCP tools carry a raw `inputJSONSchema` used verbatim while Zod tools go through `zodToJsonSchema` (`openclaude/src/utils/api.ts:208`).
- **codex** collapses spec and runtime into one trait: `ToolExecutor` pairs `spec()` with `handle()`, with default `supports_parallel→false` and `exposure→Direct` (`codex/codex-rs/tools/src/tool_executor.rs:49`). `ToolSpec` is a serde enum (`Function{name,description,parameters}` / `Freeform`) that *is* the Responses-API tool object (`codex/codex-rs/tools/src/tool_spec.rs:17`).
- **goose** derives the schema from the Rust param struct via `schemars::schema_for!` (`goose/.../developer/mod.rs:84`) and declares tools with `Tool::new(name, description, schema).annotate(ToolAnnotations::...)` (`:101`). The `read_only_hint` annotation — not a separate flag — drives auto-approval (`goose/.../permission/permission_inspector.rs:44`).

Validation-failure-as-result is universal: opencode's `wrap` decodes args and maps a decode failure to `InvalidArgumentsError`, whose message becomes the tool result (`opencode/.../tool/tool.ts:99,121`); codex returns `FunctionCallError::RespondToModel(...)` for bad args (vs `Fatal` which aborts) (`codex/codex-rs/core/src/tools/registry.rs:124,457,488`); claude-code returns an `InputValidationError` tool_result with `is_error:true` (`claude-code/.../toolExecution.ts:615,670`).

Registry as a flat list + merge. claude-code's `getAllBaseTools()` is a literal array gated by env/feature flags (`claude-code/src/tools.ts:193`); `assembleToolPool` sorts built-ins, appends MCP tools, and `uniqBy('name')` so built-ins win (`claude-code/src/tools.ts:345`). opencode builds its ~14 built-ins the same way — `invalid, shell, read, glob, grep, edit, write, task, fetch, todo, ...` (`opencode/.../tool/registry.ts:218`). codex diverges: tools are registered per turn into a router rather than a static array.

Result truncation / disk-spill bounds context growth before any summarization. claude-code spills any result over `tool.maxResultSizeChars` to a file and gives the model a preview + path (`claude-code/src/Tool.ts:466`,

`.../toolResultStorage.ts:193`); `Read` sets it to `Infinity` to avoid a Read→file→Read loop. opencode auto-truncates output past `MAX_LINES=2000` / `MAX_BYTES=50KB` to a temp file and tells the model to Grep/Read it (`opencode/.../tool/truncate.ts:15,130,139`).

Read-only drives two things. Beyond auto-approval, the read-only flag partitions execution: consecutive concurrency-safe tools run in parallel while writers serialize (claude-code `toolOrchestration.ts`, cap 10; codex takes an `RwLock` read-guard for parallel tools and a write-guard for serial ones, `codex/codex-rs/core/src/tools/parallel.rs:117`).

Scenario

The model emits a `tool_use` block: `{name:"write", input:{path:".git/config", content:"..."}}`.

1. **Render** (earlier): `write` was sent to the model as `{name:"write", description:"Write content to a file...", input_schema:{path:string, content:string}}` via `zodToJsonSchema`.
2. **Validate**: `inputSchema.safeParse({path,content})` succeeds — both fields present and typed. (Had `content` been missing, the executor would return `<tool_use_error>InputValidationError: content is required</tool_use_error>` and the loop would continue, letting the model retry.)
3. **Gate**: `write.checkPermissions({path:".git/config"})` hits the bypass-immune safety floor — writes under `.git/` always `ask`, even in bypass mode — so the gate blocks for a user decision. The user denies.
4. **Result**: instead of aborting, the executor emits `{type:tool_result, tool_use_id:..., content:"Permission denied.", is_error:true}`. The pairing invariant holds (every `tool_use` got a `tool_result`), so the next model call won't 400.
5. **Continue**: the model sees the denial, apologizes, and proposes editing a different file — self-correction with zero loop-level special-casing.

Had the write succeeded and produced a 200 KB confirmation blob (it won't, but a `bash` cat would), step 4 would instead map the output to a block, notice it exceeds `maxResultSizeChars`, write it to disk, and hand back a preview plus the file path.

Lean take

- Build the four-field tool — `{name, description, inputSchema, call}` — plus `isReadOnly` and an optional `checkPermissions`. Keep the schema in the same file as `call` so they cannot drift; codex's single `spec()`+`handle()` trait is the cleanest expression of this. Skip the ~30 UI/telemetry members entirely.
- Use a `buildTool` factory with fail-closed defaults (`isReadOnly→false`, `checkPermissions→allow`). It makes every tool safe-by-default and a few lines to declare.
- Make schema-validation failure, permission denial, and thrown exceptions **all** return an `is_error` tool result. This is the single highest-leverage decision in the chapter: it gives the loop one happy path and lets the model repair itself.
- Ship the irreducible set: `read`, `write`, `edit`, `bash`, `glob`, `grep`. Every repo has exactly these; codex's minimum is literally shell + apply_patch.
- The minimalist datapoint: **pi.dev ships only four** — `read`, `write`, `edit`, `bash` — on the thesis that the model already knows the coding task, so anything else is "situational and opt-in." No MCP/subagents/todos in-core; every extra is a user-loaded extension the agent can even write for itself (Ch. 17). If in doubt, start at four and grow by demonstrated need.
- The registry is an array, not a plugin bus. Convert each schema to JSON Schema once, sort by name for cache stability, merge externals with built-ins winning.
- Cap each result's size and spill the overflow to a file with a preview + pointer. This bounds context cheaply and defers the need for compaction. Derive schemas from your param type (zod / schemars) rather than hand-writing

JSON Schema.

4. The Model & Provider Layer

The provider layer is the seam between your agent loop and the LLM API. It builds one request, opens a streaming connection, parses server-sent events (SSE) into an internal event union, assembles tool calls from incremental JSON fragments, and retries or falls back when the network or the model misbehaves. Everything above it (the loop, tools, compaction) should be provider-agnostic.

How it works

A turn is one streaming HTTP request. You send the full message history, a system prompt, and the tool schemas; the server streams back a sequence of typed events that you fold into assistant message blocks. The defining detail of the Anthropic Messages API is that **content arrives as indexed blocks built up by deltas**, and a tool call's arguments arrive as a *string* of incremental JSON fragments (`input_json_delta`) that is only valid JSON once the block closes.

The request body for the Anthropic Messages API is small: `model`, `messages`, `system`, `tools`, `tool_choice`, `max_tokens`, and `stream: true` (`claude-code/src/services/api/claude.ts:1700`). Each tool entry is `{ name, description, input_schema }`, where `description` is the model-facing prompt and `input_schema` is JSON Schema derived from a Zod schema via `zodToJsonSchema`, or a raw `inputJSONSchema` for MCP tools (`claude-code/src/utils/api.ts:157`).

The stream parser keeps a sparse array `contentBlocks[index]`. Events mutate the block at their `index`: `content_block_start` seeds it (a `tool_use` block starts with `input: ''` — an empty *string*, not an object), `content_block_delta` appends, and `content_block_stop` finalizes. The crucial asymmetry: text deltas append to a string that is always usable, but tool-input deltas append raw JSON characters that are a syntactically-incomplete fragment until the block closes. So you **never** `JSON.parse` mid-stream — you accumulate, then parse exactly once at block close.

Flow

1. Build the request: `messages` (full or windowed history), `system`, `tools` (each `{name, description, input_schema}`), `max_tokens`, `stream: true`.
2. Open the SSE stream; `message_start` carries the message envelope and initial `usage`.
3. For each `content_block_start`, seed `contentBlocks[index]` by type. A `tool_use` block stores its `input` as an empty string accumulator.
4. For each `content_block_delta`, dispatch on `delta.type`: `text_delta` appends to `.text`; `input_json_delta` appends `partial_json` to the `.input` string; `thinking_delta`/`signature_delta` append to the thinking block.
5. On `content_block_stop`, finalize the block — and for `tool_use`, `JSON.parse` the accumulated input string (with a safe fallback to `{}` on parse failure).
6. `message_delta` carries the final `usage` and the real `stop_reason`; write them back onto the already-emitted assistant message. `message_stop` ends the stream.
7. On a transient error, retry with backoff; on repeated overload, swap to a fallback model and re-issue the whole request.

```
parse_stream(sse):
 blocks = {}            # index -> partial block
 for event in sse:
   switch event.type:
     message_start:        msg = event.message; usage = msg.usage
     content_block_start:
       if event.block.type == "tool_use":
         blocks[event.index] = { ...event.block, input: "" }   # STRING accumulator
```

```
    else:
      blocks[event.index] = { ...event.block, text: "" }
  content_block_delta:
    b = blocks[event.index]
    switch event.delta.type:
      text_delta:        b.text  += event.delta.text
      input_json_delta:  b.input += event.delta.partial_json # raw JSON chars
  content_block_stop:
    b = blocks[event.index]
    if b.type == "tool_use":
      b.input = safe_json_parse(b.input) or {}   # PARSE ONLY HERE
    emit_block(b)
  message_delta:    usage = event.usage; stop_reason = event.delta.stop_reason
  message_stop:     done()
```

Why parse only at close: a partial like `{"path": "/et` is not valid JSON, and the empty-object fallback at close means a truncated stream degrades to "tool got empty args" (which validation rejects gracefully) rather than crashing the parser.

In the wild

All five hand-roll or delegate the same accumulate-then-parse pattern; they differ in how special-cased tool calls are and how many providers they speak.

Repo	Wire format	Tool-call assembly
claude-code	Anthropic Messages	accumulate `input_json_delta`, parse at stop
openclaude	Anthropic (canonical)	same, per-provider SSE translated inward
codex	OpenAI Responses only	not special-cased; whole item then router
goose	per-format state machine	accumulate `partial_json`, parse at stop
opencode	Vercel AI SDK	SDK owns SSE; only normalize to events

- **claude-code** appends `input_json_delta.partial_json` to `contentBlock.input` during the stream (`claude-code/src/services/api/claude.ts:2111`), and at `content_block_stop` emits the block through `normalizeContentFromAPI` (`claude-code/src/utils/messages.ts:2651`) which `safeParseJSON`s the accumulated string and falls back to `{}` (`claude-code/src/utils/messages.ts:2677`). The real `stop_reason` arrives later on `message_delta` and is mutated back onto the emitted message (`claude-code/src/services/api/claude.ts:2242`).
- **goose** runs an explicit SSE state machine in `response_to_streaming_message` (`goose/crates/goose-providers/src/formats/anthropic.rs:733`). An `InputJsonDelta` pushes `partial_json` into a per-tool `args` string (`anthropic.rs:868`); `content_block_stop` parses it with `parse_tool_arguments` (`anthropic.rs:903`). Its `Provider::stream` trait returns a `MessageStream` (`goose/crates/goose-providers/src/base.rs:386`), and `complete` is just `collect_stream(stream)` (`base.rs:394`).
- **codex** does *not* special-case tool calls in the SSE layer. `kind` strings map to a typed `ResponseEvent` enum (`codex/codex-rs/codex-api/src/common.rs:74`, parsed in `sse/responses.rs:315`); tool calls arrive as ordinary `OutputItemDone` items and a separate router extracts them. This cleanly splits "parse the stream" from "interpret tool calls".

- **openclaude** keeps the whole codebase Anthropic-shaped and runs per-provider translators that *re-emit Anthropic events*. `openaiStreamToAnthropic` (`openclaude/src/services/api/openaiShim.ts:1998`) accumulates OpenAI `delta.tool_calls` into a per-call `jsonBuffer` and synthesizes `content_block_start` / `input_json_delta` / `content_block_stop`. It even recovers tool calls weak local models emit as plain text or XML (`openaiShim.ts:1578`, `:1680`).
- **opencode** delegates SSE *and* tool dispatch to the Vercel AI SDK and only normalizes the SDK's stream parts into its own event union — lowest effort, highest dependency.

The internal event union. Two shapes recur. claude-code/openclaude keep the Anthropic stream-event shape *as* the internal union (so non-Anthropic providers translate inward to it). codex defines its own `ResponseEvent` enum with rich variants (`Completed { token_usage, end_turn }`, `OutputItemDone`, `OutputTextDelta`, `RateLimits`) decoupled from any one wire format (`codex/codex-rs/codex-api/src/common.rs:74`). Either way, the loop above the parser sees one event type, never raw SSE.

Retry and fallback. claude-code's `withRetry` (`claude-code/src/services/api/withRetry.ts:170`) is a `for (attempt = 1..maxRetries+1)` generator. It honors a `retry-after` header, else uses exponential backoff `BASE_DELAY_MS * 2^(attempt-1)` capped at 32s with 25% jitter (`withRetry.ts:530`). 529 ("overloaded") is capped at `MAX_529_RETRIES = 3` and dropped immediately for background sources. Model fallback is a *separate* mechanism: on repeated 529 with a `fallbackModel` set, it throws `FallbackTriggeredError` (`withRetry.ts:347`) which the agent loop catches to swap `currentModel = fallbackModel` and re-issue the whole request. codex backs off similarly, preferring a server-requested delay, and reuses the turn-scoped session across retries to keep sticky routing; after exhausting retries it switches transport (WebSocket→HTTPS) before giving up (`codex/codex-rs/core/src/responses_retry.rs:22`). goose classifies transient errors with a `RetryConfig`; opencode uses an Effect retry policy keyed per provider.

Scenario

The model decides to read a file. Request goes out with `stream: true` and the `Read` tool schema. The SSE stream returns:

```
message_start         { usage: {input_tokens: 4120, output_tokens: 0}, stop_reason: null }
content_block_start   index=0 { type: "text" }
content_block_delta   index=0 text_delta "I'll read that file."
content_block_stop    index=0                          -> emit text block
content_block_start   index=1 { type: "tool_use", id: "toolu_01", name: "Read",
                                input: "" }
content_block_delta   index=1 input_json_delta '{"file_'
content_block_delta   index=1 input_json_delta 'path": "/e'
content_block_delta   index=1 input_json_delta 'tc/hosts"}'
content_block_stop    index=1     -> JSON.parse('{"file_path":"/etc/hosts"}') = {ok}
message_delta         { stop_reason: "tool_use", usage: {output_tokens: 38} }
message_stop
```

What the parser does: at index=1's `content_block_start` it stores `blocks[1] = {type:"tool_use", id:"toolu_01", name:"Read", input:""}`. Each delta appends raw characters — after the second delta `input` is `'{"file_path": "/e'`, which is *not* parseable, so no parse is attempted. At `content_block_stop` the accumulated `'{"file_path": "/etc/hosts"}'` parses to `{file_path:"/etc/hosts"}`. The assistant message is emitted with `stop_reason: null`; the real `stop_reason: "tool_use"` lands on `message_delta` and is written back. The loop sees a `tool_use` block present (not the `stop_reason`), so it executes `Read`, appends a `tool_result`, and re-issues the next turn. Had the stream truncated mid-arguments, the close-time parse would fail and fall back to `{}`, surfacing as an input-validation error the model can correct — not a crash.

Lean take

- **Hand-roll one provider's SSE accumulator.** It is ~150 lines: a `blocks[index]` map, a `switch` on event type, and a `switch` on delta type. Target the Anthropic Messages shape — indexed blocks make assembly trivial.

- **Accumulate tool-call JSON as a string; parse exactly once at block close**, with an empty-object fallback. Never parse a partial. This single rule eliminates a whole class of mid-stream crashes.
- **Define one internal event union and translate every provider inward to it.** Copy openclaude's seam (one canonical shape, per-transport translators) rather than branching the loop on provider. Skip multi-provider entirely until you need a second model.
- **Trust the content, not `stop_reason`.** Drive the loop off the *presence of a `tool_use` block*; treat `message_delta`'s `stop_reason` as advisory usage/metadata.
- **Retry is exponential backoff + `retry-after`, plus one fallback-model swap.** Cap overload retries low. Implement model fallback as a thrown signal the loop catches, not as branching inside the parser.
- **Skip:** WebSocket transport, non-streaming fallback, sticky-session reuse, Bedrock/Vertex field-stripping, and weak-model text/XML tool-call recovery — all real, all deferrable until a concrete provider forces them.

5. Prompt Construction & Caching

Every turn re-sends the system prompt, the tool schemas, and the full message history. Prompt construction decides *what goes where* — and the layout is driven almost entirely by one constraint: the provider caches a prefix, so anything that changes between turns must live behind everything that does not.

How it works

The request to the model has three top-level regions, and the cache treats them as an ordered hierarchy: `tools → system → messages`. A cache breakpoint hashes the cumulative prefix up to and including the marked block; a later request reads that entry only if the prefix matches byte-for-byte. Change a block at or before the breakpoint and the hash differs — a cache miss, and a re-write of everything after the change. Changing a tool definition invalidates *all three* regions; changing the system prompt invalidates system + messages but not tools.

That single rule dictates the whole design:

- **Tool descriptions travel in the `tools` field, not the prose.** They are the first cache region, so they must be the most stable bytes in the request. claude-code memoizes each tool's serialized schema for the whole session so a feature-flag flip or a `tool.prompt()` recompute can't churn those bytes (`claude-code/src/utils/api.ts:136-150`).
- **The system prompt is split into a static cacheable prefix and a dynamic suffix.** Identity, tone, tool-usage rules, and task guidance go first and never change within a session. Env info, memory, output style, MCP instructions go after a boundary marker. Only the prefix carries a `cache_control` marker.
- **Per-session and per-turn facts go even further back — into the message array.** Memory (`CLAUDE.md`/`AGENTS.md`) and the date are injected as a leading `<system-reminder>` *user* message, so editing memory mid-session never touches the cached system prefix.

Flow

1. Build the `tools` array: for each tool, `description = await tool.prompt(...)`, `input_schema = zodToJsonSchema(tool.inputSchema)`; cache the result per tool for the session (`claude-code/src/utils/api.ts:160,171`).
2. Build the system prompt as an ordered `string[]`: static sections, then a `DYNAMIC_BOUNDARY` sentinel, then dynamic sections (`claude-code/src/constants/prompts.ts:560-576`).
3. At request time, split the array at the boundary into `{text, cacheScope}` blocks; attach `cache_control: ephemeral` only to blocks whose scope is non-null (`claude-code/src/utils/api.ts:321`, `claude-code/src/services/api/claude.ts:3213`).
4. Append session-stable system *context* (git status) to the tail of the system prompt (`appendSystemContext`, `claude-code/src/utils/api.ts:437`).
5. Prepend per-session *user* context (memory + date) as a `<system-reminder>` user message at the front of the message array (`prependUserContext`, `claude-code/src/utils/api.ts:449`).

```
request layout (cache hierarchy: tools -> system -> messages)

 tools:    [ {name, description=tool.prompt(), input_schema} ... ]   <- session-stable
 system:   [ STATIC PREFIX                                    ]  <- cache_control: ephemeral
           |  identity / tone / tool-usage / tasks |       (the cached prefix)
           [ ---- DYNAMIC_BOUNDARY (dropped) ----- ]
           [ env, memory*, output-style, mcp       ]  <- no cache marker
           [ + appended systemContext: git status  ]
 messages: [ <system-reminder> user msg: CLAUDE.md + date ]  <- per-session, uncached
           [ ...real conversation..., tool_results ]
           [ + per-turn reminder attachments       ]
```

```
 * memory may be a dynamic system section AND/OR the user-message reminder.
```

The `<system-reminder>` is a plain convention, not an API feature: text wrapped in that tag inside a user message or tool result. The system prompt tells the model these tags are injected by the system and "bear no direct relation" to the message they ride in (`claude-code/src/constants/prompts.ts:131-133`). The memory reminder adds "this context may or may not be relevant... do not respond to it unless highly relevant" to curb over-anchoring (`claude-code/src/utils/api.ts:463-469`).

In the wild

Repo	System prompt assembly	Where dynamic facts go
claude-code	static prefix + `DYNAMIC_BOUNDARY` + dynamic sections `prompts.ts:560`	memory+date as `<system-reminder>` user msg `api.ts:449`; git appended to system `api.ts:437`
openclaude	same boundary marker + sections `prompts.ts:115,557`	`prependUserContext` `<system-reminder>` `api.ts:499`; memory also a dynamic section `prompts.ts:491`
opencode	per-model prompt + `input.system` + user system, joined `request.ts:58`	`<env>` block (model/cwd/git/date) as a system part `system.ts:63`
codex	`BaseInstructions.text` sent in Responses `instructions` field `client.rs:849,877`	AGENTS.md as a `user` item; env as a context item, not in instructions
goose	MiniJinja render of `system.md` `prompt_manager.rs:164`	hints/extras appended as `# Additional Instructions` `prompt_manager.rs:193`

Divergences worth noting:

- **Cache marker placement.** claude-code/openclaude attach `cache_control` only to the pre-boundary blocks; the post-boundary join gets `cacheScope: null` (`claude-code/src/utils/api.ts:392-396`). The boundary is the breakpoint.
- **System as a field, not a message.** codex puts base instructions in the Responses `instructions` field (`client.rs:877`) — outside the message list entirely. In its `responses_lite` path it instead injects them as a leading `developer`-role message (`client.rs:835-846`). opencode does the field-form only for OpenAI OAuth (`request.ts:99`), otherwise it maps each system string to a `role: "system"` message (`request.ts:105-110`).
- **Templating.** goose is the only one that templates the prompt: MiniJinja over `system.md` with a typed context struct (`prompt_template.rs:3,85`). Everyone else concatenates strings.
- **Cache-stable ordering as an explicit goal.** goose sorts extensions by name before rendering "for multi session prompt caching" (`prompt_manager.rs:128-129`); opencode collapses the system array to exactly two blocks (header + rest) so plugins can't fragment the prefix (`request.ts:74-78`). Same instinct as claude-code's boundary.

Scenario

Turn 1 of a session in claude-code. `getSystemPrompt(tools, model)` returns a `string[]`: `[intro, system-rules, doing-tasks, actions, tool-usage, tone, efficiency, __DYNAMIC_BOUNDARY__, session-guidance, memory, env_info, ...]` (`prompts.ts:560-576`). `getUserContext()` discovers `CLAUDE.md` by walking cwd and returns `{claudeMd, currentDate}` (`context.ts:170-187`); `getSystemContext()` snapshots git status once (`context.ts:124`).

At request time `splitSysPromptPrefix` finds the boundary index, joins blocks 0..n before it into one `{cacheScope:'global'}` block and blocks after it into one `{cacheScope:null}` block (`api.ts:362-404`). `buildSystemPromptBlocks` stamps `cache_control: {type:'ephemeral'}` on the global block only (`claude.ts:3213`). `appendSystemContext` tacks git status onto the system tail; `prependUserContext` inserts the `<system-reminder>` user message carrying `CLAUDE.md` + date ahead of the user's first real message (`api.ts:437,449`).

Turn 2: the user sends a follow-up. The static system prefix and the tool schemas are byte-identical, so the provider returns a cache hit for the tools region and the system prefix — only the new user message and tool results are fresh input. The user edits `CLAUDE.md` mid-session: because memory rides in a *user* message (after system in the hierarchy), the system-prefix cache still hits; only the messages region re-writes. Had memory been in the static system prefix, that edit would have invalidated system *and* every cached message after it.

Lean take

- **Three regions, one breakpoint.** Put a single `cache_control` marker at the end of your static system prefix. Keep tool schemas and the prefix byte-stable for the whole session; that is 90% of the caching win for ~10 lines of code.
- **Small beats clever.** Caching makes a big prompt *cheap to resend*, not *good* — a bloated system prompt still crowds the model's attention and buries the task. The spread in the wild is enormous: pi.dev ships **under 1,000 tokens**, opencode ~15k, claude-code ~55k. Start tiny (identity + tool-usage rules + a memory hook) and add a line only when a real failure demands it; every addition is a permanent tax on every turn.
- **Tool descriptions belong in the `tools` field.** Never inline them in the prose. They are the first and most cache-sensitive region; one edit invalidates the whole request.
- **Push volatile facts as far back as the hierarchy allows.** Env/date/git after a boundary in the system suffix; memory and per-turn context as `<system-reminder>` user messages. Anything that changes within a session must sit behind the breakpoint.
- **`<system-reminder>` is just a tagged string + one sentence of system-prompt framing.** Adopt the convention (and the "may not be relevant" hedge); skip building any special machinery.
- **Skip, until you need them:** per-model prompt variants, MiniJinja templating, multi-scope (`global`/`org`) cache tiers, the 1-hour TTL, and a second breakpoint for long conversations. Add the second breakpoint only once history routinely pushes the first one past the 20-block lookback window.

6. Context Management & Compaction

Every turn resends the whole conversation, so the message list grows until it threatens the model's context window. Context management is the subsystem that keeps the next request under that ceiling: estimate the size, decide *when* to shrink, and shrink *how* — ideally without losing information the user can still see.

How it works

There are four separable concerns, and the repos that do this well keep them separate.

1. A token budget. You need a cheap, synchronous estimate of how many tokens the agent-visible history will cost. The honest finding across these codebases is that a real tokenizer is overkill: codex estimates with a flat `bytes / 4` heuristic (`approx_token_count`, ceiling division) and never tokenizes at all (`codex/codex-rs/utils/string/src/truncate.rs:71`). claude-code and goose do count, but the count only feeds a threshold comparison where a 10-20% error is irrelevant because there is a multi-thousand-token safety buffer underneath.

2. When to compact. Two triggers, and you want both:

- *Proactive*: at the top of each turn, if `estimate > threshold`, compact before calling the model. The threshold is `contextWindow − buffer`, expressed either as an absolute reserve (claude-code: `effectiveWindow − 13k`, `claude-code/src/services/compact/autoCompact.ts:72`) or a ratio (goose: usage ratio `> 0.8`, `goose/crates/goose/src/context_mgmt/mod.rs:188`).
- *Reactive*: the estimate can be wrong, or a single huge tool result can blow the window mid-turn. So the loop must also catch a real `context-length-exceeded` provider error, compact, and retry — bounded by an attempt cap so you don't spin (goose: cap 2, `goose/crates/goose/src/agents/agent.rs:2372`; claude-code's reactive path withholds the 413 from the consumer until recovery fails).

3. How to compact. Ask the model to summarize the transcript, then *rebuild* the history from a few fixed pieces rather than appending the summary. codex rebuilds `[initial_context] + [recent user msgs ≤ 20k tokens] + [summary]` (`codex/codex-rs/core/src/compact.rs:585`); claude-code rebuilds `[boundaryMarker, ...summary, ...preservedTail, ...attachments]` (`claude-code/src/services/compact/compact.ts:330`). The summary prompt is the real lever — claude-code's structured template has the model fill nine fixed sections (intent, technical concepts, files & code, errors/fixes, problem solving, all user messages, pending tasks, current work, optional next step) inside a `<summary>` block (`claude-code/src/services/compact/prompt.ts:61`).

4. Don't delete — hide. The cleanest idea is goose's **dual visibility**: every message carries `user_visible` and `agent_visible` flags. Compaction marks the old messages `agent_invisible` (still user-visible) and the summary `agent_only` (`goose/crates/goose/src/context_mgmt/mod.rs:141`); only `is_agent_visible()` messages are sent to the model (`goose/crates/goose/src/agents/reply_parts.rs:269`). The transcript stays whole for the user and replay is clean.

Below summarization sit **cheaper tiers** you reach for first, because they cost no extra model call: cap each tool result's size and spill the overflow to a file pointer + preview (claude-code `maxResultSizeChars`, `claude-code/src/Tool.ts:466`), and a non-LLM prune that erases stale tool-output *content* by id once enough recent tokens are protected (opencode, `opencode/packages/opencode/src/session/compaction.ts:242`).

Flow

1. Top of turn: compute `estimate = approxTokens(agentVisibleMessages)`.
2. If `estimate > contextWindow − buffer`: run the summarizer (one model call over the transcript).
3. Rebuild history: `[initial context] + [recent user messages ≤ N tokens] + [summary]`; preserve the latest user message verbatim.

4. Mark the compacted-away messages agent-invisible; mark the summary agent-only. Resume the loop on the rebuilt history.
5. If instead the model call returns `context-length-exceeded`: same compaction, then retry — up to an attempt cap, after which surface the error.
6. Independently, on every tool result: if `len > cap`, spill to disk and replace with a preview + pointer. Optionally prune stale tool outputs without an LLM.

```
BUFFER          = 13_000            # absolute reserve, OR use ratio 0.8
RECENT_BUDGET   = 20_000            # tokens of trailing user msgs to keep verbatim

function maybeCompact(messages):
   if approxTokens(agentVisible(messages)) <= contextWindow - BUFFER:
       return messages                        # under budget, no-op
   return compact(messages)

function compact(messages):
   summary  = model.summarize(agentVisible(messages))   # one extra call
   initial  = initialContext(messages)                  # system/env, re-derivable
   recent   = takeFromEnd(userMessagesOf(messages), RECENT_BUDGET)  # newest-first
   rebuilt  = initial + recent + [summary]
   for m in messages: m.agentVisible = false            # hide, don't delete
   summary.userVisible = false                          # summary is model-only
   return rebuilt

# in the loop:
try:
   resp = model.call(system, agentVisible(messages), tools)
except ContextLengthExceeded:
   if (attempts += 1) > CAP: raise                      # bounded reactive recovery
   messages = compact(messages); continue
```

In the wild

Repo	Trigger / budget	Compaction strategy
claude-code	`effWindow − 13k` proactive + reactive 413 withhold `autoCompact.ts:72`	cheapest-first cascade → 9-section summary; `[bndry,summary,tail]` `compact.ts:330`
openclaude	same `AUTOCOMPACT_BUFFER 13k` `autoCompact.ts:77`	5-layer cascade ending in same summary
codex	`bytes/4` heuristic; tokens `> limit` `truncate.rs:71`, `turn.rs:886`	one pass: `[init]+[recent≤20k]+[summary]` `compact.rs:585`
goose	usage ratio `> 0.8` `context_mgmt:188`; reactive cap 2 `agent.rs:2372`	LLM summary + dual-visibility flags `context_mgmt:141`
opencode	`usable = limit.input − reserved` `overflow.ts:10`	hidden compaction agent + non-LLM `prune` `compaction.ts:242`

Divergences worth noting:

- **Tokenizer vs heuristic.** Only codex fully commits to the byte heuristic (`approx_token_count = ceil(bytes/4)`); the count exists only to compare against a threshold, so precision is wasted effort (`codex/codex-rs/core/src/context_manager/history.rs:519`).
- **Reserve as absolute vs ratio.** claude-code/openclaude subtract a fixed `13_000`-token buffer (and separately reserve up to `20k` output tokens for the summary itself, `autoCompact.ts:33`); goose uses a 0.8 *ratio* of the limit.

Both reduce to "leave headroom."

- **Recent-window selection.** codex walks user messages newest-first, accumulating until it hits the 20k budget, truncating the boundary message if it overflows (`codex/codex-rs/core/src/compact.rs:598`). goose instead preserves only the single most-recent text-only user message verbatim (`goose/crates/goose/src/context_mgmt/mod.rs:112`).
- **Cheaper tiers.** claude-code stacks tool-result disk-spill → snip → microcompact → context-collapse → autocompact, most behind feature flags (`claude-code/src/query.ts:365`). opencode's `prune` is the leanest: walk back, protect ~25% of usable tokens worth of recent tool calls, then mark older completed tool outputs `compacted` — no model call (`opencode/packages/opencode/src/session/compaction.ts:242`). goose has a "middle-out" variant that removes tool responses from the *middle* of the transcript outward, keeping head and tail context (`goose/crates/goose/src/context_mgmt/mod.rs:242`).
- **Don't-delete.** goose is the only one with first-class dual visibility; codex and claude-code achieve non-destructiveness for the *user* via persistence (boundary marker / replacement history) rather than per-message flags.

Scenario

A long refactoring session on a 200k-window model with goose's defaults (threshold 0.8, so an effective ceiling of 160k agent-visible tokens).

1. Turn 41 begins. The agent-visible history — dozens of `read`/`grep` tool results plus assistant reasoning — estimates at 168k tokens. `check_if_compaction_needed` computes `168k / 200k = 0.84 > 0.8` and returns true (`context_mgmt/mod.rs:232`).
2. `compact_messages` finds the most recent text-only user message ("now wire it into the CLI") and marks it for verbatim preservation (`context_mgmt/mod.rs:112`). It calls the model with the summarization prompt over the full agent-visible slice.
3. The model returns a structured summary: files touched, the bug fixed in step 30, the pending CLI-wiring task. History is rebuilt: every old message gets `with_agent_invisible()`, the summary becomes `agent_only`, and a continuation message plus the preserved user message follow (`context_mgmt/mod.rs:141`).
4. The next request now carries ~12k agent-visible tokens. The user's transcript UI still shows all 41 turns, because those messages are still `user_visible` — nothing was destroyed.
5. Turn 45: a single `cat` of a 900k-character generated file slips through the estimate and the provider returns `ContextLengthExceeded`. The loop's error arm increments `compaction_attempts` to 1 (under the cap of 2), compacts again, replaces the conversation, and breaks to retry the turn (`agent.rs:2372`). Had it failed twice, the agent would surface a "context limit still exceeded" message instead of looping forever.

Lean take

- **Ship the byte heuristic.** `ceil(bytes/4)` over the agent-visible messages is enough to drive a threshold (codex). Add a real tokenizer only if it visibly misfires — it almost never will, because of the buffer.
- **Two triggers, one compaction function.** Check `estimate > window − buffer` at the top of the loop *and* catch `context-length-exceeded` inside it; both call the same `compact()`. Cap reactive retries (2 is fine) so a too-big single message can't spin.
- **One summarize-and-rebuild pass.** `[initial] + [recent user msgs ≤ ~20k] + [summary]`, latest user message preserved verbatim. A single structured-summary prompt covers the real cases; claude-code's 9-section template is the upgrade if you want fidelity. Skip the 5-layer cascade.
- **Hide, don't delete.** Add `agentVisible`/`userVisible` flags from day one and filter the model's view by `agentVisible` (goose). It costs almost nothing and makes compaction non-destructive and replay correct.

- **Cap and spill tool results before you ever summarize.** A per-tool size cap with disk-spill-to-pointer (claude-code `Tool.ts:466`) is the cheapest tier and removes the most common cause of blowups; an optional non-LLM prune of stale tool outputs (opencode) buys more runway with no model call.
- **Skip** middle-out thinning, per-tool-pair summarization, context-collapse commit-logs, and remote compaction backends until a real workload demands them.

7. Permissions, Approval & Sandboxing

The approval gate is the function that runs *before* every side-effecting `tool.call()` and returns `allow | ask | deny`. It is the agent's one chance to stop a destructive action before it happens, to ask a human, or to refuse — and a denial must feed back to the model as a result, never abort the turn. OS sandboxing is a separate, optional layer underneath it.

How it works

The gate sits between "the model asked to run tool X with input Y" and "the side effect happens". Two design questions define it:

> **Where do the rules live, and how do they combine?** Two families. A **fixed-precedence pipeline** runs a hardcoded sequence of checks (deny rules → ask rules → the tool's own `checkPermissions` → a bypass-immune safety floor → the permission *mode*), first decisive result wins (claude-code `permissions.ts:1071`, `:1158`). An **ordered rule list** is flatter: one list of `{permission, pattern, action}`, **last wildcard match wins**, default `ask` (opencode `permission/index.ts:28`). The pipeline is more rigid but bakes in a safety floor; the rule list is tiny and composable.
>
> **Who owns tool-specific logic?** A general gate can't know that `git status` is safe but `rm -rf` is not. So a tool exposes its own `checkPermissions(input)` that the gate calls. For `bash` this means parsing the command into sub-commands and deciding per sub-command (claude-code splits the AST and combines results; opencode tree-sitter-parses and asks per real sub-command, `shell.ts:257`).

Three invariants every repo shares: **the gate runs before the side effect**; a **deny becomes an `is_error` tool result fed back to the model**, not an exception that kills the turn (claude-code `toolExecution.ts:1034`, codex `parallel.rs:76`); and a **safety floor** for the agent's own config (`.git/`, `.claude/`, shell rc files) prompts even when the user has turned all prompting off.

Flow

1. Model emits `tool_use {id, name, input}`; the loop validates `input` against the tool schema (schema failure is itself an `is_error` result, not a gate call).
2. Gate checks **deny** first — an explicit deny rule short-circuits to `deny`.
3. Gate checks **ask** rules and the **tool's `checkPermissions`** (e.g. bash per-sub-command allow/ask/deny).
4. Gate checks the **safety floor**: writes to protected paths force `ask` even under bypass.
5. Gate applies the **permission mode** (default / acceptEdits / bypass / plan): mode can only *relax* toward allow for things the floor didn't catch.
6. Result: `allow` → run the tool; `ask` → block for a human (a persistent "always" answer appends a new allow rule); `deny`/rejection → synthesize an `is_error` tool result and **continue the loop**.
7. (Optional) If allowed and the tool shells out, run it inside an OS sandbox.

```
function decide(tool, input, ctx) -> allow | ask | deny:
   # ---- fixed-precedence pipeline (claude-code permissions.ts:1071) ----
   if denyRuleFor(tool, input):              return deny     # 1. explicit deny wins
   if askRuleFor(tool):                      return ask      # 2. blanket ask rule
   r = tool.checkPermissions?(input)         # 3. tool owns its risk logic
   if r == deny:                             return deny     #    (bash sub-cmd denied)
   if r == ask and r.fromExplicitRule:       return ask      #    content ask rule
   if isProtectedPath(input):                return ask      # 4. SAFETY FLOOR (immune)
   if ctx.mode in (bypass, planWithBypass):  return allow    # 5. mode can relax
   if alwaysAllowRuleFor(tool, input):       return allow
   return (r == passthrough) ? ask : r       # default: ask
```

```
# ---- alternative: ordered rule list, last-match-wins (opencode index.ts:28) ----
function evaluate(permission, pattern, rules) -> rule:
   return rules.flat()
               .findLast(r => match(permission, r.permission)
                           and match(pattern, r.pattern))
          ?? { action: "ask" }               # default ask if nothing matches

# ---- the gate's contract at the call site (claude-code toolExecution.ts:1034) ----
d = decide(tool, input, ctx)
if d == ask:   d = await ctx.askUser(tool, input)   # "always" -> append allow rule
if d == deny:  return toolResult(id, "denied", isError=true)   # FEED BACK, no abort
out = tool.call(input, ctx)                          # the only path to a side effect
```

In the wild

Two rule models, an orthogonal sandbox axis, and a universal "deny is a result" rule.

Repo	Rule model	Safety floor / sandbox
claude-code	fixed precedence `permissions.ts:1071`	bypass-immune paths; bwrap/Seatbelt
openclaude	same precedence + `fullAccess` `permissions.ts:1089`	bypass-immune paths; bwrap
opencode	ordered list, last-match-wins `index.ts:28`	none (ask-only)
codex	two gates: execpolicy + mode `safety.rs:32`	real OS sandbox `manager.rs:60`
goose	inspector pipeline `agent.rs:629`	none default; opt-in container

- **claude-code / openclaude — fixed precedence + bypass-immune floor.** `checkRuleBasedPermissions` runs deny → ask → `tool.checkPermissions` → safety, and is the *subset bypass mode respects* (`permissions.ts:1071`, openclaude `:1089`). The safety floor is concrete: `DANGEROUS_FILES` (`.gitconfig`, `.bashrc`, `.claude.json`, …) and `DANGEROUS_DIRECTORIES` (`.git`, `.vscode`, `.claude`) in `filesystem.ts:57,74`, enforced by `checkPathSafetyForAutoEdit` (`filesystem.ts:620`) which returns a `safetyCheck` decision that survives even `bypassPermissions` mode (`permissions.ts:1255`). Modes: `default`/`acceptEdits`/`bypassPermissions`/`plan` (`types/permissions.ts:17`); openclaude adds `fullAccess`.
- **opencode — ordered list, last-match-wins.** `evaluate` flattens all rulesets and takes the *last* matching wildcard rule, default `ask` (`index.ts:28`). `ask` blocks on a `Deferred`; a `"always"` reply appends `{action:"allow"}` rules at runtime and auto-resolves other pending asks now covered (`index.ts:145`). No safety floor, no OS sandbox — gating is purely ask-before-side-effect.
- **codex — two orthogonal gates.** *Whether to run* = execpolicy rule engine + `AskForApproval` mode (`Never`/`OnRequest`/`UnlessTrusted`/`Granular`) → `SafetyCheck::{AutoApprove, AskUser, Reject}` (`safety.rs:20,32`). *How to run safely* = pick an OS sandbox: `get_platform_sandbox` returns Seatbelt (macOS) / LinuxSeccomp / WindowsRestrictedToken (`manager.rs:60`), and `transform` **rewrites the argv** — prepending the `sandbox-exec` binary on macOS (`manager.rs:379`) or re-exec'ing the Codex binary as a Landlock+seccomp helper on Linux (`manager.rs:406`). Enforcement needs no in-process trust.
- **goose — inspector pipeline + annotation-driven SmartApprove.** An ordered list of `ToolInspector`s (Security → Egress → Adversary → Permission → Repetition, `agent.rs:629-646`) each returns Allow/Deny/RequireApproval. The `PermissionInspector` keys off `GooseMode` (Chat/Auto/Approve/SmartApprove, `goose_mode.rs:24`): in SmartApprove a tool's `read_only_hint` annotation auto-allows it, and unknown tools are classified by an LLM read-only detector (`permission_inspector.rs:159,171`). Reads auto-allow, writes gate, zero per-tool config.

Deny → result, never abort is unanimous. claude-code builds a `tool_result` with `is_error:true` and pushes it as a user message (`toolExecution.ts:1034`), openclaude the same (`:1147`), opencode throws `Rejected`/`Corrected` that becomes the tool result, codex maps every non-`Fatal` error (including `RespondToModel`) to a `failure_response` tool result and only `Fatal` aborts (`parallel.rs:75-76`), goose emits a placeholder tool response on `Deny`.

Scenario

Bypass mode is on; the model calls `bash` with `cat README.md && rm -rf .git`.

```
input  = { command: "cat README.md && rm -rf .git" }   ; mode = bypassPermissions
1. schema validates -> ok
2. bash.checkPermissions parses the AST into sub-commands:
    "cat README.md"  -> known-safe read    -> allow
    "rm -rf .git"    -> matches Bash(rm:*)? no rule -> passthrough
                     -> ".git" hits DANGEROUS_DIRECTORIES (filesystem.ts:74)
                     -> checkPathSafetyForAutoEdit -> safetyCheck (filesystem.ts:620)
3. combine: any sub-command not 'allow' downgrades the whole call
    -> decisionReason.type == 'safetyCheck'  (permissions.ts:1147)
4. mode check: bypassPermissions would allow... BUT step 4 ran BEFORE step 5,
  and safetyCheck is bypass-immune -> stays 'ask'         (permissions.ts:1255)
5. gate result = ask -> human prompted "rm -rf .git — approve?"
6a. human rejects -> tool_result(is_error=true, "Permission denied") (toolExecution.ts:1034)
   -> appended as user message; loop CONTINUES; model sees the denial and adapts.
```

The key beat: even with *all prompting disabled*, the write to `.git` still prompts, because the safety floor is checked before the mode that would have waved it through.

Lean take

- **Start with opencode's model: one ordered `{permission, pattern, action}` list, last-match-wins, default `ask`** (`index.ts:28`). It is ~10 lines, composes by concatenation, and "always" is just appending an allow rule. Skip a fixed-precedence pipeline until you actually need a bypass mode.
- **Give each tool a `checkPermissions(input)` hook** so risky tools own their logic. For `bash`, split on `&&`/`|`/`;` and prefix-match each segment against allow/deny lists; deny if any segment denies, ask if any is unknown. Tree-sitter (opencode `shell.ts:257`) is nicer but a regex split covers the 90% case.
- **Add a bypass-immune safety floor** as a hardcoded check: writes to `.git/`, your agent's own config dir, and shell rc files always `ask`, regardless of mode (claude-code `filesystem.ts:57,74`). This is the single highest-value safety feature and costs a dozen lines.
- **Auto-allow read-only tools** via an `isReadOnly` flag so reads never prompt (goose SmartApprove `permission_inspector.rs:159`).
- **Make deny a result, not an abort** — synthesize an `is_error` tool_result and continue the loop (claude-code `toolExecution.ts:1034`, codex `parallel.rs:76`). Same for schema-validation and tool exceptions. One happy path, no exception-driven control flow.
- **Treat OS sandboxing as a separable, later layer.** opencode and goose ship with none and are useful. If you add one, copy codex's argv-rewrite approach (`manager.rs:379,406`) so enforcement needs no in-process trust — but ship the ask-before-side-effect gate first.

8. Hooks

Hooks are user-configured shell commands (or HTTP/LLM-prompt actions) that the agent runtime fires at fixed lifecycle points — before/after a tool runs, when a prompt is submitted, when the turn stops. They let a user *observe*, *block*, or *modify* the agent's behavior without touching its code: a synchronous side channel into the loop, governed by exit codes and JSON on stdout.

How it works

A hook is matched by **event** (when it fires) and **matcher** (a sub-filter, usually a tool-name pattern). On the event, the runtime serializes a JSON *hook input* describing what is about to happen, spawns each matching command with that JSON on **stdin**, and interprets the command's **exit code** and **stdout** as a decision:

- **exit 0** — success. stdout may be JSON (a structured decision) or plain text (injected as context, depending on event).
- **exit 2** — blocking error. stderr is fed back to the model as the reason; the pending action is denied (PreToolUse) or the turn is forced to continue (Stop).
- **other non-zero** — non-blocking error; logged/shown, action proceeds.

For richer control the command can print a JSON object on stdout: `{"decision": "block","reason":...}` or, per-event, `hookSpecificOutput` carrying a `permissionDecision` (`allow`/`deny`/`ask`), an `updatedInput` (rewrite the tool args), or `additionalContext` (text injected into the conversation). JSON overrides the exit code. The two channels are equivalent for the common deny-with-reason case; JSON exists for the modify/allow/ask cases that an exit code can't express.

Config lives in `settings.json` keyed by event → array of `{matcher, hooks[]}` groups; each hook is `{type:"command", command, timeout?, if?}` (claude-code also has `prompt`/`agent`/`http` types). Multiple matched hooks run **in parallel**; their results are aggregated, with any block winning.

Flow (PreToolUse, the load-bearing case)

1. Model emits a `tool_use`. Before the permission gate, the runtime builds a `PreToolUse` hook input: `{hook_event_name, tool_name, tool_input, tool_use_id, session_id, cwd, ...}`.
2. It collects hooks registered for `PreToolUse` whose `matcher` (regex / pipe-list / `*`) matches `tool_name`, plus any `if:` rule filter.
3. Each command is spawned via the shell with the input JSON on stdin and a per-hook timeout.
4. Each result is parsed: exit 2 → block; stdout JSON → permission decision / `updatedInput` / `additionalContext`; else plain success.
5. Results aggregate. A `deny` short-circuits: the tool never runs; an error `tool_result` carrying the reason goes back to the model. `updatedInput` rewrites the args that reach `tool.call()`. `additionalContext` is appended.
6. If allowed, the normal permission gate and `tool.call()` proceed; afterward `PostToolUse` fires the same way (it can inject context or rewrite MCP output, but cannot un-run the tool).

```
on PreToolUse(tool_name, tool_input, tool_use_id):
 hooks = matchingHooks("PreToolUse", tool_name)         # matcher + if-filter
 results = parallel(for h in hooks: spawn(h, stdin=JSON(input), timeout=h.t))
 agg = { behavior: "allow", updatedInput: tool_input }
 for r in results:                                       # any block wins
   if r.exit == 2:                      agg.behavior="deny"; agg.reason=r.stderr
   elif json := parse(r.stdout):
     if json.decision == "block":       agg.behavior="deny"; agg.reason=json.reason
     d = json.hookSpecificOutput
     if d.permissionDecision:           agg.behavior = d.permissionDecision
     if d.updatedInput:                 agg.updatedInput = d.updatedInput
```

```
    if d.additionalContext:             agg.contexts += d.additionalContext
if agg.behavior == "deny":  return errorToolResult(agg.reason)   # tool skipped
input = agg.updatedInput                                  # feeds tool.call()
proceed to permission gate -> tool.call()
```

In the wild

claude-code and openclaude are the same codebase (openclaude line numbers differ; they carry the full feature set). codex and goose are leaner Rust ports that adopt the same wire protocol (exit-2 / `decision:block`).

Aspect	claude-code / openclaude	codex (Rust)	goose (Rust)
Events	27 (full set)	~10	8
Config	`settings.json` matchers	`hooks.toml`	plugin `hooks.json`
Action types	command/prompt/agent/http	command	command
Block channel	exit 2 + JSON	exit 2 + JSON	exit 2 + JSON

- **Event set (claude-code):** `HOOK_EVENTS` lists 27 events incl. `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `UserPromptSubmit`, `Stop`, `StopFailure`, `SubagentStart`/`SubagentStop`, `Notification`, `PreCompact`/`PostCompact`, `SessionStart`/`SessionEnd`, `PermissionRequest`/`PermissionDenied`, `Setup`, `FileChanged`, `CwdChanged`, `WorktreeCreate`/`Remove` — `claude-code/src/entrypoints/sdk/coreTypes.ts:25`.
- **Config schema:** `HooksSchema` = partial record of event → `HookMatcher[]`, where `HookMatcher = {matcher?, hooks: HookCommand[]}` and the command hook is `{type:"command", command, if?, shell?, timeout?, ...}` — `claude-code/src/schemas/hooks.ts:194,211` (openclaude `src/schemas/hooks.ts:194,211`).
- **Spawn:** command hooks run via the shell with input JSON written to stdin and newline-terminated — `claude-code/src/utils/hooks.ts:977,1210` (openclaude `:1166,1203`). PowerShell uses an explicit argv path.
- **Matcher semantics:** `*`/empty = match all; `[a-zA-Z0-9_|]+` is exact or pipe-list; anything else is a regex — `claude-code/src/utils/hooks.ts:1346`. The `if:` field is permission-rule syntax (e.g. `Bash(git *)`) evaluated against tool input before spawning — `claude-code/src/schemas/hooks.ts:19`, `:1390` (`prepareIfConditionMatcher`).
- **Decision parsing:** exit 2 → `outcome:"blocking"` with stderr as reason (`claude-code/src/utils/hooks.ts:2648`); JSON `decision:"approve"|"block"` maps to allow/deny (`:525`); `hookSpecificOutput.permissionDecision` for PreToolUse maps allow/deny/ask and pulls `updatedInput`/`additionalContext` (`:593-622`).
- **Parallel + aggregate:** all matched hooks run via `all(hookPromises)`; results fold into one `AggregatedHookResult` — `claude-code/src/utils/hooks.ts:2143,2744`.
- **PreToolUse consumer:** the tool executor drains `runPreToolUseHooks`, applies `hookPermissionResult` and `hookUpdatedInput`, and on `stop` returns a tool-result error without calling the tool — `claude-code/src/services/tools/toolExecution.ts:800-862,921`.
- **Stop consumer:** when the model emits no tool_use, `handleStopHooks` runs; a blocking error rebuilds `State` with the reason appended and `continue`s the loop (forcing the agent to keep working) — `claude-code/src/query.ts:1267-1306`.
- **codex:** TOML config `HookEventsToml` with `MatcherGroup{matcher?, hooks[]}` and command handlers — `codex/codex-rs/config/src/hook_config.rs:36,133,143`; `PreToolUseHookResult::{Continue{updated_input}, Blocked(reason)}` mirrors the modify/deny split — `codex/codex-rs/core/src/hook_runtime.rs:55,198-216`.

- **goose:** plugin `hooks/hooks.json`, regex matcher, command actions; `HookManager::emit_blocking` returns `HookDecision::{Allow, Deny}` and `deny_reason` checks exit code 2 *or* `{"decision":"block"}` — `goose/crates/goose/src/hooks/mod.rs:50,224,364,429`. Non-blocking events use `emit`. Wired in `agents/agent.rs:1052-1066` (PreToolUse gate). Key divergence: goose stops at the **first** deny (`:377`), claude-code runs all and folds.

Scenario

A user wants to forbid the agent from editing `.env` files. They add to `settings.json`:

```
{ "hooks": { "PreToolUse": [
 { "matcher": "Edit|Write",
   "hooks": [ { "type": "command",
                "command": "jq -e '.tool_input.file_path|endswith(\".env\")' \
                           >/dev/null && { echo blocked >&2; exit 2; } || exit 0" } ] }
] } }
```

1. Model emits `Edit{file_path:".env", ...}`. Executor builds the `PreToolUse` input and matches `Edit` against `"Edit|Write"` → hit (`hooks.ts:1346`, pipe-list path).
2. The command spawns with the input JSON on stdin (`hooks.ts:977,1210`). `jq` sees `file_path` ends in `.env`, prints `blocked` to stderr, exits 2.
3. `execCommandHook` returns `status:2`; the result becomes `{outcome:"blocking", blockingError:{reason:"blocked"}}` (`hooks.ts:2648`).
4. Aggregation sets `permissionBehavior:"deny"`; `resolveHookPermissionDecision` yields a deny, the executor never calls `Edit`, and a `tool_result` error ("Edit hook error: blocked") returns to the model (`toolExecution.ts:921`).
5. The model reads the denial and routes around it (e.g. asks the user). Had the hook instead printed `{"hookSpecificOutput":{"hookEventName":"PreToolUse", "permissionDecision":"allow","updatedInput":{...}}}`, the executor would have rewritten the args and let the edit through.

Lean take

- Ship three events first: **PreToolUse** (deny/modify — the only one that changes outcomes), **PostToolUse** (observe / inject context), and **Stop** (force-continue). UserPromptSubmit and SessionStart are cheap follow-ons.
- Adopt the **exit-2 + `{"decision":"block","reason"}`** protocol verbatim — goose does this in ~450 lines (`hooks/mod.rs`) and it interops with existing hook scripts. Pass the event payload as JSON on stdin; read stdout/exit.
- Config = `event -> [{matcher, hooks:[{command, timeout}]}]`. Matcher: treat `*` as all, a plain string / `A|B` as exact, else regex (`hooks.ts:1346`). Skip the permission-rule `if:` filter until you need it.
- Run matched hooks in parallel; **any block wins**; fold `updatedInput` and `additionalContext`. First-deny-wins (goose) is simpler and fine.
- Make hooks **fail-open**: spawn errors, timeouts, and non-2 exits must NOT block the agent (goose `:362`) — a broken hook should degrade, not brick the loop.
- Skip for v1: prompt/agent/http action types, async/background hooks, plugin/ skill sourcing, workspace-trust gating, the 20+ niche events. All are product surface, not mechanism.

9. Memory & Context Files

Persistent project/user instructions the agent loads from disk on every turn — `CLAUDE.md`, `AGENTS.md`, `.goosehints` — plus the directives that pull other files inline. This is how an agent "remembers" your conventions across sessions without any of it living in the model.

How it works

A context file is a markdown file the agent discovers by walking the filesystem, concatenates into one blob, and feeds the model every turn. Three independent decisions define the mechanism:

- **Discovery hierarchy** — which files, in what precedence. Universally: *enterprise/managed* (machine-wide policy) < *user* (`~/.claude`, all projects) < *project* (checked-in, cwd walked up to root) < *local* (private, gitignored). Later = higher priority because the model weights later text more (claude-code `claudemd.ts:1-10`).
- **Embeds** — a file can pull in others. claude-code/openclaude use `@path` (`claudemd.ts:18-25`); goose uses the same `@path` token with an *import boundary* (`import_files.rs:9`). Both cap recursion depth and dedup by path.
- **Injection** — *where* the blob lands. Two camps: a leading **`<system-reminder>` user message** prepended to history (claude-code, openclaude, codex), versus a **system-prompt section** ("hints") baked into the system string (goose, opencode). User-message injection survives prompt-cache prefix reuse better and is easy to mark `isMeta`; system injection keeps it out of the transcript.

The discovery is a memoized directory walk. Files closer to cwd load *later* (higher priority). `@`-includes are spliced *before* the including file so the parent's text wins on conflict.

Flow

1. Resolve roots: managed dir (`/etc/...`), user home (`~/.claude`), then collect every directory from cwd up to filesystem root.
2. For each root in priority order (managed → user → project root → … → cwd), try the known filenames (`CLAUDE.md`, `.claude/CLAUDE.md`, `.claude/rules/*.md`, `CLAUDE.local.md`). Skip already-seen paths.
3. For each file read: strip frontmatter/HTML comments, scan leaf text nodes for `@path` tokens, recurse into each (depth-capped, cycle-guarded), splice children before parent.
4. Tag each entry with its type (Managed/User/Project/Local) and render with a provenance label + the override instruction header.
5. Join into one string; inject as a leading user `<system-reminder>` (or a system-prompt section). Memoize the whole walk; bust the cache on `/memory` edit, compaction, or worktree change.

```
discover(cwd):
 roots = [managed_dir, user_home]
 d = cwd; while d != fs_root: roots.push(d); d = parent(d)
 # project/local walk goes root -> cwd so nearer files load later (win)
 out = []; seen = {}
 for root in [managed, user] + reverse(roots_cwd_to_root):
   for name in filenames_for(root):           # CLAUDE.md, .claude/rules/*.md ...
     out += process(join(root, name), seen)
 return out

process(path, seen):
 if path in seen or depth > MAX: return []
 seen.add(path)
 raw = read(path) or return []
 body = strip_frontmatter(strip_html_comments(raw))
 children = []
 for inc in extract_at_includes(body):        # @path in leaf text nodes only
```

```
  children += process(resolve(inc, dir(path)), seen)   # spliced BEFORE parent
return children + [{path, type, body}]
```

In the wild

Repo	Files / hierarchy	Injection point
claude-code	Managed/User/Project/Local + `.claude/rules/*.md` `claudemd.ts:1`	user `<system-reminder>` `api.ts:449,463`
openclaude	same 4 tiers + TeamMem/AutoMem `claudemd.ts:1177`	user `<system-reminder>` `api.ts:513`
codex	`AGENTS.md` root→cwd + `AGENTS.override.md` `agents_md.rs:13,39`	user item `<INSTRUCTIONS>` `user_instructions.rs:18`
goose	`.goosehints`/`AGENTS.md`/`CLAUDE.md`, global+local `load_hints.rs:10,46`	system-prompt `hints` extra `prompt_manager.rs:174`
opencode	`AGENTS.md`/`CLAUDE.md`, first match wins `instruction.ts:61,83`	system section, `Instructions from:` `instruction.ts:127`

Divergences worth copying or avoiding:

- **claude-code is the richest hierarchy.** Six types (`Managed`, `User`, `Project`, `Local`, plus feature-gated `AutoMem`/`TeamMem`), each path resolved by `getMemoryPath` (`config.ts:1779`): `~/.claude/CLAUDE.md`, `<cwd>/CLAUDE.md`, `<cwd>/CLAUDE.local.md`, `<managed>/CLAUDE.md`. Project discovery walks cwd→root and loads root→cwd so nearer files win (`claudemd.ts:854,878`). The render labels provenance inline — `(project instructions, checked into the codebase)` vs `(user's private global instructions...)` (`claudemd.ts:1168`) — so the model knows whose rule it is.
- **`.claude/rules/*.md` with conditional globs.** Rule files carry frontmatter `paths:` globs; an unconditional rule loads always, a conditional one only when a tool touches a matching path (lazy, nested-directory load via `getMemoryFilesForNestedDirectory` `claudemd.ts:1249`, glob match `processConditionedMdRules:1354`). This is the closest thing to "subdir lazy load" in the TS repos.
- **`@`-include is a lexer pass, not a regex.** claude-code lexes markdown (`marked`) and only extracts `@path` from **text/list leaf nodes**, skipping code spans and fenced blocks (`claudemd.ts:452`). Accepts `@x`, `@./x`, `@~/x`, `@/abs`; depth cap 5 (`:537`); circular refs blocked by a `seen` set; external (outside-cwd) includes require explicit approval. Non-text extensions are rejected to keep binaries out of context.
- **codex caps total bytes and concatenates root→cwd.** It finds the project root by `.git` marker (`find_nearest_ancestor_with_markers`), concatenates every `AGENTS.md` from root down with a `--- project-doc ---` separator (`agents_md.rs:43,322`), and truncates at `project_doc_max_bytes` (`:89,115`). `AGENTS.override.md` is a local, gitignored override (`:39,224`).
- **goose is the only true lazy subdir loader.** Eager load is global (`~/.config/goose` + `~/.agents/AGENTS.md`) plus git-root→cwd (`load_hints.rs:54,89`). A `SubdirectoryHintTracker` then watches every tool's `path`/`command` args (`:42`), and on the next turn `load_new_hints` reads `.goosehints`/`AGENTS.md` from any **newly-touched subdir** under cwd and appends it as a `subdir_hints:<dir>` system extra (`:72,109`). Hints land in the system prompt, not the transcript (`prompt_manager.rs:174`).
- **opencode dedups hard: first project match wins.** It does not stack `AGENTS.md` from every ancestor — the first existing file up the tree ends the search (`instruction.ts:83-93`), plus one global file and config-listed extra paths/URLs, each rendered `Instructions from: <path>`.

The `/memory` editing command (claude-code `commands/memory/memory.tsx`) is a thin file picker: it lists the discoverable memory paths, `mkdir -p`s the `~/.claude` dir if needed, creates the file with `wx` (no-clobber), opens `$EDITOR`/`$VISUAL` (`:42`), then **clears the memoize cache** so the next turn re-reads (`call:83-89`). It is not a tool the model calls — it is human-driven.

Persistent memory dirs are a separate system from instruction files. Both forks add a write-as-you-go store the *agent* maintains:

- claude-code/openclaude AutoMem: `~/.claude/memory/MEMORY.md` index + date-named daily logs + topic files (`memdir/paths.ts:85,93,257`), injected as an extra memory type, truncated to line+byte caps.
- openclaude SessionMemory: a background **forked subagent** periodically writes `<projectDir>/<sessionId>/session-memory/summary.md` of the live conversation (`SessionMemory/sessionMemory.ts:1`, path `filesystem.ts:289`) — non-blocking, per-session.
- openclaude TeamMem: `<base>/projects/<root>/memory/team/MEMORY.md`, synced across the org with a secret-scanner guard before write (`teamMemPaths.ts:92`, `teamMemorySync/`).

Scenario

You run `claude` in `~/work/api/services/billing/`, a git repo rooted at `~/work/api/`.

1. `getUserContext()` calls `getMemoryFiles()` (`context.ts:171`). The walk collects roots: managed dir, `~/.claude`, then `~/work/api`, `~/work/api/services`, `~/work/api/services/billing` (`claudemd.ts:854`).
2. It reads in priority order: `~/.claude/CLAUDE.md` ("use bun, be concise"), then root→cwd — `~/work/api/CLAUDE.md` ("monorepo: kebab-case routes"), then `~/work/api/services/billing/CLAUDE.md` ("this service owns Stripe webhooks"). The billing file contains `@./CONVENTIONS.md`; the lexer finds the token in a list item (`:452`), resolves it, and splices `CONVENTIONS.md` *before* the billing file (`:664`).
3. Each entry is rendered `Contents of <path> (project instructions, checked into the codebase):\n\n<body>` and joined under the header `Codebase and user instructions are shown below... These instructions OVERRIDE any default behavior` (`:90,1194`).
4. `getClaudeMds` returns the blob into `userContext.claudeMd` (`context.ts:186`). On the model call, `prependUserContext` wraps it in a single leading `<system-reminder>` user message marked `isMeta` (`api.ts:449`) ahead of the real conversation.
5. The model now sees, before your first prompt: bun preference, monorepo conventions, the Stripe ownership note, and the spliced `CONVENTIONS.md` — the nearer billing rules visually last, so they win on any conflict.

Lean take

- **Build the walk and one injection point.** cwd→root collection, load root→cwd so nearer wins, three tiers (user `~/.agent/AGENT.md`, project `AGENT.md`, local `AGENT.local.md`). Inject as a single leading user message with a "these override defaults; may be irrelevant" header. ~100 lines.
- **Add `@include` early — it pays for itself.** A regex over non-code text, resolve relative to the file's dir, a `seen` set, a depth cap of ~5. Skip the full markdown-lexer sophistication until users actually put `@x` inside code fences.
- **Memoize the walk; bust on edit.** One cache keyed on cwd; clear it when your `/memory` command writes a file or when you compact. Re-reading disk every turn is the bug, not the feature.
- **`/memory` is a file picker + `$EDITOR`, not a tool.** Don't let the model edit instruction files via a tool call; make it a human command that `mkdir -p`s, no-clobber-creates, opens the editor, and invalidates the cache.
- **Skip until needed:** managed/enterprise tier, conditional `rules/*.md` globs, team sync, and per-session auto-memory subagents. They are product surface, not mechanism. Adopt goose's lazy subdir reload (track tool path args, load that dir's hints next turn) *only* if your projects are large monorepos.

• **Pick injection by your cache strategy.** User-message injection if you cache the system-prompt prefix and want memory to vary per cwd without busting it; system-section injection (goose) if you want it out of the visible transcript.

10. Skills

A skill is an on-demand, model-invoked instruction bundle: a folder with a `SKILL.md` (YAML frontmatter + markdown body) and optional scripts/reference files. The point is **progressive disclosure** — only the skill's name and one-line description sit in context up-front; the heavy body loads only when the model decides to use it, so a hundred skills cost a few hundred tokens, not the whole context window.

How it works

A skill folder is `<root>/skills/<skill-name>/SKILL.md`. The frontmatter carries metadata; the body is the prompt that gets injected on invocation:

```
---
name: pdf                         # optional display name (dir name is the default)
description: Extract and fill PDF forms   # the ALWAYS-in-context hint
when_to_use: when the user has a PDF...    # optional extra match signal
allowed-tools: Bash, Read, Write           # tool scoping for the skill turn
disable-model-invocation: false            # if true, only a human can run it
context: inline                            # "inline" | "fork" (subagent)
model: sonnet                              # optional per-skill model override
---
<markdown body — workflow steps, references to ./scripts/foo.py, etc.>
```

Discovery runs at startup: walk a fixed set of roots (managed/policy, user, project, `--add-dir`), read each `<dir>/SKILL.md`, parse frontmatter, and build a lightweight `Command` record. The body is read into memory but **never sent** until invocation — at startup only `name + description (+ when_to_use)` are estimated for token cost. The full set of metadata is then formatted into a single budgeted listing and injected as a `<system-reminder>` user message: "The following skills are available for use with the Skill tool: …". That listing is the up-front half of progressive disclosure.

When the model wants a skill it calls the `Skill` tool with `{skill, args}`. The tool validates the name, runs the permission gate, then loads the body: it reads the SKILL.md, prepends `Base directory for this skill: <dir>`, substitutes `$ARGUMENTS` / `${CLAUDE_SKILL_DIR}` / `${CLAUDE_SESSION_ID}`, runs any inline shell (`!`...``) blocks, and returns the expanded text as **new user messages** appended to the conversation. The model reads those next turn and follows the instructions. That is the on-demand half.

Flow

1. **Startup discovery.** Walk skill roots; for each `<name>/SKILL.md`, parse frontmatter into a `Command` (name, description, allowed-tools, context, model, body-in-memory). Dedup by canonical path (realpath) so symlinks / overlapping roots don't double-list.
2. **Up-front listing.** Each turn, diff loaded skills against an already-sent set; format the *new* ones as `- name: description` within a ~1% char budget; inject as a `skill_listing` system-reminder. Body excluded.
3. **Model selects.** Seeing the listing + the `Skill` tool description, the model emits `tool_use{ Skill, {skill:"pdf", args:"form.pdf"} }`.
4. **Validate + gate.** Tool checks the name exists, isn't `disable-model-invocation`, then evaluates deny/allow rules → ask the user if unknown.
5. **Load body.** Read SKILL.md, prepend base-dir header, expand args/vars/inline-shell. Apply `allowed-tools` scoping for the skill's turn.
6. **Inject + continue.** Return the expanded body as new user messages (inline) or run it in a forked subagent (`context: fork`) and return its result. The loop iterates; the model now has the full instructions.

```
# Pseudocode: discovery + progressive disclosure + invocation

# --- at startup (once) ---
skills = {}
```

```
for root in [managed, user, project, *addDirs]:
 for dir in listdir(root/"skills"):
   fm, body = parse_frontmatter(read(dir/"SKILL.md"))
   skills[dir.name] = Skill(name=dir.name, desc=fm.description,
                            allowed_tools=fm["allowed-tools"], body=body,
                            ctx=fm.context, base_dir=dir)   # body kept, NOT sent

# --- each turn, before calling the model ---
new = [s for s in skills.values() if s.name not in sent]
if new:                                      # only names+descriptions, budget-capped
 listing = "\n".join(f"- {s.name}: {s.desc}" for s in fit_budget(new))
 messages.append(system_reminder(f"Skills available:\n{listing}"))
 sent |= {s.name for s in new}

# --- Skill tool .call({skill, args}) ---
s = skills[skill]
if s.disable_model_invocation: return error("not model-invocable")
if not permit("Skill", skill): return ask_or_deny
text = f"Base directory for this skill: {s.base_dir}\n\n{s.body}"
text = expand(text, args, CLAUDE_SKILL_DIR=s.base_dir, SESSION_ID=sid)
text = run_inline_shell(text)                # !`...` blocks, with allowed_tools scope
return { new_messages: [user(text)],         # inline: body enters context now
        context_modifier: scope_tools(s.allowed_tools) }
```

In the wild

The Claude-Code lineage (`claude-code`, `openclaude`) implements skills as an explicit `Skill` tool that injects the body inline. Codex has **no skill tool**: it lists skills in a developer message and instructs the model to *read the SKILL.md path itself* with its file tools. Opencode has a `skill` tool that returns the body wrapped in `<skill_content>` tags.

Concern	claude-code / openclaude	codex / opencode
Invocation	`Skill` tool, body injected	codex: read path; opencode: `skill` tool
Up-front cost	`name + desc` listing	name + desc + path/locator
Body load	new user messages	codex: model `Read`s file
Scoping	`allowed-tools` per turn	opencode: permission per skill

- **Format / frontmatter.** Fields parsed in `claude-code/src/skills/loadSkillsDir.ts:185` (`parseSkillFrontmatterFields`): `description`, `name`, `allowed-tools`, `when_to_use`, `model`, `disable-model-invocation`, `user-invocable`, `context`, `agent`, `effort`, `paths`, `hooks`. Codex frontmatter is just `name`/`description` (`codex/.codex/skills/code-review/SKILL.md:1`); opencode adds `slash` (`opencode/packages/core/src/skill.ts:33`).
- **Discovery / dir layout.** Only `skill-name/SKILL.md` directory form is accepted (`claude-code/src/skills/loadSkillsDir.ts:407`), roots walked in `getSkillDirCommands` (`:638`), deduped by `realpath` (`getFileIdentity`, `:118`). Opencode globs `{*.md,**/SKILL.md}` (`opencode/packages/core/src/skill.ts:79`). Codex loads via its own `SkillsService` (`codex/codex-rs/core/src/skills.rs:17`).
- **Progressive disclosure — the listing.** Built by `formatCommandsWithinBudget` and capped at ~1% of the context window in chars, per-entry `MAX_LISTING_DESC_CHARS = 250` (`claude-code/src/tools/SkillTool/prompt.ts:21,29,70`). Injected as a `skill_listing` attachment (`src/utils/attachments.ts:2661`, `:2741`) rendered to "The following skills are available…" (`src/utils/messages.ts:3728`). Token estimate uses frontmatter only (`estimateSkillFrontmatterTokens`, `loadSkillsDir.ts:100`). Codex uses a 2% budget (`codex/codex-rs/core-skills/src/render.rs:19`) with an explicit truncation warning when it overflows (`:24`).

- **The Skill tool.** `claude-code/src/tools/SkillTool/SkillTool.ts:331`, input `{skill, args}` (`:291`), tool name `"Skill"` (`SkillTool/constants.ts:1`). Tool description tells the model the listing lives in system-reminders and invocation is a "BLOCKING REQUIREMENT" (`SkillTool/prompt.ts:173-195`). Openclaude is identical (`openclaude/src/tools/SkillTool/SkillTool.ts:331`).
- **Body load + substitution.** `createSkillCommand.getPromptForCommand` prepends the base-dir header, substitutes `$ARGUMENTS` / `${CLAUDE_SKILL_DIR}` / `${CLAUDE_SESSION_ID}`, then runs inline shell — except for MCP skills, which are untrusted and skip shell execution (`claude-code/src/skills/loadSkillsDir.ts:344-398`).
- **allowed-tools scoping.** On inline invocation, `call()` returns a `contextModifier` that merges the skill's `allowedTools` into `alwaysAllowRules` for that turn (`claude-code/src/tools/SkillTool/SkillTool.ts:775-806`). Opencode asserts a per-skill permission before loading (`opencode/.../tool/skill.ts:76`).
- **Permission default.** Unknown skills default to `ask`; skills with only "safe" frontmatter properties auto-allow (`SAFE_SKILL_PROPERTIES`, `SkillTool.ts:529-538,875`); deny/allow rules support `name` and `prefix:*` matching (`ruleMatches`, `:451`).
- **Inline vs fork.** `context: 'fork'` runs the skill as an isolated subagent with its own token budget via `runAgent` (`claude-code/src/tools/SkillTool/SkillTool.ts:122,622`); `inline` (default) injects into the main conversation.
- **Dynamic / conditional discovery.** claude-code re-scans nested `.claude/skills` dirs as files are touched (`discoverSkillDirsForPaths`, `loadSkillsDir.ts:861`) and activates `paths:`-gated skills only when a matching file is edited (`activateConditionalSkillsForPaths`, `:997`).
- **MCP / remote skills.** Skills can come from MCP servers (`skill://` URIs, `openclaude/src/skills/mcpSkills.ts:14`) or remote stores; codex exposes `skills.list`/`skills.read` for orchestrator-owned skills (`codex/.../core-skills/src/render.rs:30`); opencode downloads a remote `index.json` + files into a cache dir (`.../skill/discovery.ts:99`).

Scenario

A repo has `.claude/skills/pdf/SKILL.md` with frontmatter `description: Fill and extract PDF forms`, `allowed-tools: Bash, Read, Write`, and a body that references `./scripts/fill.py`.

1. **Startup.** `getSkillDirCommands` reads the file, parses frontmatter into a `Command{name:"pdf", description:"Fill and extract PDF forms", allowedTools: ["Bash","Read","Write"], body:<…>}`. The body (~3K tokens) stays in memory.
2. **Turn 1.** Before the model call, `getSkillListingAttachments` sees `pdf` is unsent, formats `- pdf: Fill and extract PDF forms` (~12 tokens), and injects it as a system-reminder. Cost in context so far: ~12 tokens, not 3K.
3. **User:** "fill out form.pdf with my address." The model matches the listing, emits `tool_use{ Skill, {skill:"pdf", args:"form.pdf"} }`.
4. **Gate.** `checkPermissions` finds no deny rule; `pdf` has only safe properties → auto-allow.
5. **Load.** `call()` runs `getPromptForCommand("form.pdf")`: prepends `Base directory for this skill: /repo/.claude/skills/pdf`, substitutes `$ARGUMENTS → form.pdf` and `${CLAUDE_SKILL_DIR}`, and returns the expanded body as a new user message. `contextModifier` scopes the turn's tools to `Bash/Read/Write`.
6. **Result.** Next iteration the model sees the full workflow, runs `Bash python ${CLAUDE_SKILL_DIR}/scripts/fill.py form.pdf`, and completes — the 3K-token body entered context only for this task.

Lean take

- **Build the folder + frontmatter + a `Skill` tool, nothing else.** Discovery = "glob `*/SKILL.md`, parse YAML, keep body in memory." Up-front = "inject `- name: desc` lines as a system message." Invocation = "tool reads the matching body and appends it as a user message." That is the entire mechanism.
- **Keep the body out of context until invoked.** This is the whole value — if you inline every skill body you have re-invented a bloated system prompt. A char/token budget on the listing is enough; skip the 1%/2% tuning.
- **Two cheap substitutions earn their keep:** a `Base directory:` header (so the body can point at `./scripts/*`) and `$ARGUMENTS`. Add `${SKILL_DIR}` only if your bodies shell out.
- **Scope tools per skill** via `allowed-tools` — one line, real safety, and it lets a skill grant `Bash(python:*)` for its turn without a global allow.
- **Default unknown skills to `ask`**, support `name` and `prefix:*` allow rules, and never run inline shell from remote/MCP skill bodies.
- **Skip for v1:** forked-subagent execution, conditional `paths:` activation, dynamic nested-dir re-scan, remote/MCP skill stores, per-skill model/effort overrides, and the resume-dedup bookkeeping. All are refinements on top of the three-line core.

11. Slash & Custom Commands

Slash commands are the CLI's command palette: text that starts with `/` is intercepted *before* the agent loop. Some are handled entirely client-side and never reach the model (`/clear`, `/model`, `/compact`); others are user-authored markdown files that **expand into a prompt** and are then injected as a user turn. This chapter is about the two kinds and the expand-then-inject machinery.

How it works

There are two command categories, and they share one dispatch table but diverge at execution. A **local command** (`type: 'local'` / `'local-jsx'`) carries a lazy `load()` returning a `call()` implementation; it runs in-process, mutates session state or renders UI, and returns messages with `shouldQuery: false` so the loop does not call the model (`claude-code/src/types/command.ts:62-152`, dispatched at `processSlashCommand.tsx:657-669`). A **prompt command** (`type: 'prompt'`) is the markdown kind: its `getPromptForCommand(args, ctx)` returns content blocks, which are pushed as a user message with `shouldQuery: true` — the model runs as if the user had typed the expanded text (`command.ts:25-57`, `processSlashCommand.tsx:869-920`).

Markdown commands are discovered by walking `.claude/commands/` (and `skills/`) from cwd up to the git root, plus the user dir `~/.claude/` and a managed/policy dir; files are read with ripgrep `--files --glob '*.md'` and YAML frontmatter is split off (`markdownConfigLoader.ts:234-289,546-600`). Each file becomes a `PromptCommand`. The **command name** is the path relative to the base dir with `/` turned into `:`, so `.claude/commands/git/commit.md` is `/git:commit` — directory **namespacing** (`loadSkillsDir.ts:523-552`).

Frontmatter fields map onto the command: `description`, `argument-hint` (gray hint in the picker), `allowed-tools` (extra tool grants for this command's turn), `model` (override; `inherit` means parent model), `arguments` (named positionals), plus `context: fork` to run as a sub-agent (`loadSkillsDir.ts:208-264`). The body is the prompt template.

At invocation `getPromptForCommand` runs three substitution passes over the body, in order: argument substitution, then `${CLAUDE_SKILL_DIR}`/`${CLAUDE_SESSION_ID}` literals, then embedded shell execution (`loadSkillsDir.ts:344-398`). `@file` mentions are *not* expanded here — they ride out as text and are turned into file attachments downstream by the same `@`-mention attachment pass that handles normal user input (`processSlashCommand.tsx:897`).

Flow

1. Input starts with `/` → `parseSlashCommand` splits `commandName` and the rest as a single `args` string (`slashCommandParsing.ts:25-60`).
2. Look up the command in the merged table (built-ins + discovered markdown + plugin/MCP). Unknown → error.
3. If `type !== 'prompt'`: lazy-`load()` and `call()` it in-process; emit result messages with `shouldQuery: false`. **Done — model never invoked.**
4. If `type === 'prompt'`: call `getPromptForCommand(args, ctx)`: a. `substituteArguments`: replace `$name`, `$ARGUMENTS[i]`, `$i`, `$ARGUMENTS`; if no placeholder matched, append `ARGUMENTS: <args>`. b. replace `${CLAUDE_SKILL_DIR}` / `${CLAUDE_SESSION_ID}`. c. `executeShellCommandsInPrompt`: run `` !`cmd` `` and ```` ```! …``` ```` , permission-checked, splice stdout in place.
5. Wrap the expanded text as a user message (plus a metadata tag message and a `command_permissions` attachment carrying `allowedTools`/`model`), set `shouldQuery: true`, hand to the loop.

```
# argument substitution core (mirrors substituteArguments + apply_skill_arguments)
substitute(body, rawArgs, argNames):
   if rawArgs is null: return body                    # not invoked with args
   parts = shell_split(rawArgs)                        # quote-aware tokens
```

```
    for i, name in argNames:                                # $name -> parts[i]
        body = replace(body, /\$name\b/, parts[i] or "")
    body = replace(body, /\$ARGUMENTS\[(\d+)\]/, parts[idx] or "")
    body = replace(body, /\$(\d+)\b/,            parts[idx] or "")  # $1..$9
    body = replaceAll(body, "$ARGUMENTS", rawArgs)          # whole string, raw
    if nothing was replaced and rawArgs:                    # graceful fallback
        body = body + "\n\nARGUMENTS: " + rawArgs
    return body

# then, only for non-MCP (trusted) sources:
for match in body.matchAll(/!`([^`]+)`/) + /```!\n(.*?)```/:
    assert permission_allows(Bash, match.command)           # gate before exec
    body = body.replace(match, run_shell(match.command).stdout)
```

In the wild

Mechanism	claude-code / openclaude	opencode	goose / codex
Built-in (no model)	`type:'local'` w/ `load()`	TUI dispatch	enum match
Markdown source dir	`.claude/commands`	`command/`, `commands/`	`.agents/skills`
`$ARGUMENTS`/`$1`	yes	yes	yes (goose)
`!`bash`` embed	yes	yes	no
`@file` embed	downstream attach	`resolvePromptParts`	no
Namespacing	dir → `a:b`	path → name	flat

Tight notes with anchors:

- **claude-code / openclaude** share the implementation verbatim. `substituteArguments` does named → `$ARGUMENTS[i]` → `$i` → `$ARGUMENTS`, then appends `ARGUMENTS:` if nothing matched (`argumentSubstitution.ts:94-145`; openclaude `argumentSubstitution.ts:124-141`). Shell embedding uses `!`cmd`` (inline) and ```! (block) regexes, each permission-checked against `BashTool` before running (`promptShellExecution.ts:49-143`; openclaude identical at `:53-110`). MCP skills are explicitly barred from shell execution as untrusted (`loadSkillsDir.ts:374-396`). openclaude's only real divergence is the `model` frontmatter resolving through a multi-provider `parseUserSpecifiedModel` (`openclaude/src/skills/loadSkillsDir.ts:222-225`).
- **opencode** loads `command/**/*.md` and `commands/**/*.md` via glob, frontmatter → config, body → `template` (`config/command.ts:12-39`). Expansion lives in the session: positional `$1..$N` (last placeholder absorbs the tail), then `$ARGUMENTS`, then `!`…`` shell, with `@file` turned into file message parts by `resolvePromptParts` (`session/prompt.ts:1371-1431`, regexes `:1591-1595`, `config/markdown.ts:5-6`). A `subtask`/agent command runs in a sub-session instead of inlining (`session/prompt.ts:1438-1450`).
- **goose** lists built-ins (`prompts`, `compact`, `clear`, `skills`, `doctor`, `goal`, `grind`, `status`) separately from disk-loaded skills, merging with builtin > recipe > skill precedence on normalized names (`slash_command.rs:30-53,68-72`). Its `apply_skill_arguments` is a faithful Rust port: one regex covers `$ARGUMENTS[i]`, `$ARGUMENTS`, `$pos`, `$name`, with the same no-placeholder `ARGUMENTS:` append (`skills/arguments.rs:6-52`). No shell/`@file` embedding.
- **codex** keeps built-ins as a `SlashCommand` enum dispatched in a `match` arm that handles `/clear`, `/compact`, `/model` locally (`tui/src/slash_command.rs:12-122`, `chatwidget/slash_dispatch.rs:233-271`). Custom prompts exist but argument handling is a TUI placeholder-fill, not a full `$ARGUMENTS`/bash/file engine — the leanest of the five.

Scenario

A repo has `.claude/commands/git/fixup.md`:

```
---
description: Stage and amend with a generated message
argument-hint: [scope]
allowed-tools: Bash(git add:*), Bash(git commit:*)
---
Current staged diff:
!`git diff --cached --stat`
Amend the last commit for scope "$1". Files touched: $ARGUMENTS
```

User types `/git:fixup auth login`.

1. `parseSlashCommand` → `{commandName:'git:fixup', args:'auth login'}` (`slashCommandParsing.ts:25`). The `:` came from the `git/` namespace at load (`loadSkillsDir.ts:545-552`).
2. Command is `type:'prompt'`; loop calls `getPromptForCommand('auth login', ctx)`.
3. `substituteArguments`: tokens `["auth","login"]`. `$1`→`auth`, `$ARGUMENTS`→`auth login` (`argumentSubstitution.ts:106-136`).
4. `executeShellCommandsInPrompt`: `` !`git diff --cached --stat` `` is permission-checked (allowed by the temporary `allowed-tools` grant) and replaced with the stat output (`promptShellExecution.ts:89-131`).
5. The expanded body is wrapped as a user message; a `command_permissions` attachment carries `Bash(git add:*)`/`Bash(git commit:*)` so the model's ensuing `git commit --amend` is auto-allowed for this turn; `shouldQuery:true` sends it (`processSlashCommand.tsx:902-916`). The model proceeds with the diff already in context.

Contrast: `/model` on the same CLI hits `type:'local-jsx'`, opens a picker, sets the model in app state, and returns with `shouldQuery:false` — no API call.

Lean take

- Build one dispatch table keyed by name. Tag each command "client" or "prompt". Client commands run a function and return; prompt commands expand and get injected as a user turn. This single split is the whole feature.
- Make markdown commands the default extension point: a directory of `*.md`, frontmatter for `description`/`model`/`allowed-tools`, body as template. Name = relative path with `/`→`:` for free namespacing. Skip a plugin system at first.
- Implement substitution in one pass with the documented precedence (named → `$i` → `$ARGUMENTS`) and the no-placeholder `ARGUMENTS:` append — it is what makes zero-placeholder commands still receive their args. Use a quote-aware splitter for `$1..$9`.
- `` !`bash` `` embedding is high-value and cheap, but **gate every embedded command through your permission check** and **never run shell from untrusted (remote/MCP) command sources** — that is the one security-load-bearing rule.
- Reuse your existing `@file` attachment path rather than special-casing it in the expander; the expanded text is just user input.
- Skip for v1: forked/subtask execution, `context:fork`, conditional `paths` activation, inode dedup, worktree fallbacks, per-turn `allowed-tools` grants. Add the tool-grant attachment only once permissions actually bite.

12. Output Styles

An output style is a swappable block of system-prompt text that changes the agent's persona and response behavior (e.g. `default` vs `Explanatory` vs `Learning`). The user picks one; it is injected near the top of the system prompt and can optionally *replace* the default coding instructions. It is a Claude-Code feature only — codex, goose, and opencode have no equivalent.

How it works

A style is a `{ name, description, prompt, source, keepCodingInstructions }` record (`claude-code/src/constants/outputStyles.ts:11-23`). Two built-ins ship in code — `Explanatory` and `Learning` — and `default` is the sentinel `null` (`outputStyles.ts:41-42`). Users add more as markdown files; each file's body becomes `prompt`, its YAML frontmatter supplies `name` / `description` / `keep-coding-instructions` (`src/outputStyles/loadOutputStylesDir.ts:34-78`).

The selected style is just a string in `settings.json` (`outputStyle`, `src/utils/settings/types.ts:639-642`). At prompt-build time `getOutputStyleConfig()` resolves that string against the merged style map and returns the config or `null` (`outputStyles.ts:206-211`). The system-prompt assembler then does two things: it injects the style body as a `# Output Style: <name>` section, and — this is the load-bearing part — it *drops the default "doing tasks" coding instructions* unless the style sets `keepCodingInstructions` (`prompts.ts:560-563`). So a style can either augment the base prompt (both built-ins keep instructions) or wholly override the agent's behavioral contract.

The intro line also flexes: with a style active the agent is told it helps users "according to your Output Style below" instead of "with software engineering tasks" (`prompts.ts:174` / `prompts.ts:180`).

Flow

1. User sets `outputStyle: "Explanatory"` in settings (via `/config`; the `/output-style` slash command is now a deprecation stub — `src/commands/output-style/output-style.tsx:3`).
2. On each system-prompt build, `getAllOutputStyles(cwd)` merges built-ins + plugin + managed + user + project markdown styles, last-writer-wins (`outputStyles.ts:137-175`).
3. `getOutputStyleConfig()` reads `settings.outputStyle`, returns the matching config (or `null` for `default` / unknown) (`outputStyles.ts:206-211`).
4. Assembler injects `getOutputStyleSection(cfg)` into the dynamic section list and conditionally includes the default coding instructions (`prompts.ts:501-502`, `prompts.ts:560-563`).
5. Result is the full system prompt sent to the model that turn.

```
resolve:  name = settings.outputStyle  (default if unset)
         cfg  = allStyles[name]        (null => "default")

assemble system prompt:
 intro    = cfg ? "helps ... according to your Output Style"
                : "helps ... with software engineering tasks"
 + system section (always)
 + DEFAULT_CODING_INSTRUCTIONS  ONLY IF cfg==null OR cfg.keepCoding==true
 + actions / tools / tone sections (always)
 + ...dynamic sections...
      "# Output Style: <name>\n<cfg.prompt>"   (omitted when cfg==null)
```

In the wild

Repo	Output styles?	Mechanism
claude-code	Yes	prompt section + instruction override
openclaude	Yes (identical)	same code, line-for-line
codex / goose / opencode	No	no persona-swap concept

- claude-code and openclaude are byte-identical here: same `OutputStyleConfig` type, same two built-ins, same loader, same injection (`openclaude/src/constants/outputStyles.ts:11-216`, `openclaude/src/constants/prompts.ts:145-152`, `:556-563`).
- Built-in styles: `Explanatory` (educational insights) and `Learning` (asks the user to hand-write `TODO(human)` snippets); both set `keepCodingInstructions: true` so they layer on top of the base prompt (`outputStyles.ts:43-134`).
- Storage / precedence: `.claude/output-styles/*.md`, loaded user-dir then project-dirs then managed/policy, with built-in lowest and project/managed highest (`outputStyles.ts:158-172`, `src/utils/markdownConfigLoader.ts:303-378`). Plugins may even *force* a style via `force-for-plugin` frontmatter (`outputStyles.ts:181-204`).
- The active style name is echoed into the `system_init` event for the status line / SDK (`src/utils/messages/systemInit.ts:55`, `:75`).
- Divergence: there is no runtime hot-swap — `getAllOutputStyles` is `memoize`d and changes "take effect on the next session" per the deprecation message (`outputStyles.ts:137`, `output-style.tsx:3`).

Scenario

User has `~/.claude/output-styles/blunt.md`:

```
---
name: Blunt
description: Terse, no preamble
keep-coding-instructions: false
---
You answer in at most three sentences. No filler, no apologies.
```

They set `outputStyle: "Blunt"` and restart. On the next turn:

1. `getOutputStyleConfig()` returns `{ name: "Blunt", prompt: "You answer...", keepCodingInstructions: false, source: "userSettings" }`.
2. Intro renders "helps users according to your Output Style below" (`prompts.ts:174`).
3. Because `keepCodingInstructions === false`, the entire `getSimpleDoingTasksSection()` block — read-before-edit, no-gold-plating, security guidance — is **omitted** (`prompts.ts:560-563`).
4. A `# Output Style: Blunt` section carrying the body is appended in the dynamic region (`prompts.ts:501-502`, `:150-151`).
5. The model now operates under the terse contract with the default coding doctrine stripped out; tool schemas and the always-on system/tools sections are unchanged.

Lean take

- Model a style as `{ name, prompt, replace?: bool }`. The whole feature is "pick a string, look up a record, splice it into the prompt." Build it in an afternoon.

- The one non-obvious bit worth copying: a `replace` flag that lets a style *delete* your default behavioral instructions, not just append. Append-only is the trivial 80%; full override is what makes a style a true persona.
- Store styles as markdown-with-frontmatter under a config dir and key the selection off one settings field. Skip plugin sources, forced styles, managed precedence, and per-turn re-resolution — read once at session start.
- Keep it distinct from your other prompt inputs: memory (CLAUDE.md) is always-on project context injected as a user-side reminder; skills are on-demand tool-triggered instructions; an output style is a single, user-chosen, session-wide persona that can rewrite the base prompt.
- Don't bother with a dedicated slash-command UI; a config field plus "applies next session" is enough and avoids cache-busting the prompt prefix mid-session.

13. Subagents & Orchestration

A subagent is not a new runtime. It is the *same* agent loop invoked recursively with a child context: its own message list, a filtered tool set, and a step/budget limit. The parent calls it through a normal tool (`Task`/`Agent`), the child runs to completion, and the child's final text becomes the parent's `tool_result`. This buys context isolation (the child's exploration never pollutes the parent window) at the cost of one recursion level — which is the 95% case.

How it works

The mechanism has four moving parts. First, a **tool** the model can call (`Task` or `Agent`) whose input is a free-text `prompt` plus a `subagent_type`. Second, an **agent definition** selected by that type — a `name`/`description`/`tools`/`model` record, usually parsed from a Markdown file's YAML frontmatter. Third, a **child context**: a cloned-but-isolated execution context with its own `agentId`, abort controller, and a tool pool filtered to what the definition allows. Fourth, the **same loop** (`query()` / `reply()`) run over the child's messages until it stops, with its final assistant text captured and returned upward.

Isolation matters most for the *window*, not the *machinery*. The child gets a fresh `Message[]` seeded only with its task prompt and system prompt; it burns its own tokens reading files and running tools; the parent sees only the summary. A step or turn limit caps the child so a runaway exploration cannot hang the parent — when the limit is hit, the loop strips tools and forces a final text answer rather than aborting.

Critically, the child's tool set usually **excludes the subagent tool itself**, so subagents cannot spawn subagents. That one exclusion is what keeps "orchestration" from metastasizing into unbounded recursion.

Flow

1. Model emits `tool_use` for `Task` with `{ subagent_type, prompt, description }`.
2. Tool resolves the `AgentDefinition` for `subagent_type` (built-in or from `.claude/agents/*.md`); error back to the model if unknown.
3. Build a child context: new `agentId`, fresh abort controller, tool pool filtered by the definition's allow/deny lists (and minus the subagent tool).
4. Seed child `Message[]` with the task prompt; pick system prompt + model from the definition (or inherit).
5. Run the *same loop* over the child context until no `tool_use` (or step/turn limit hit -> strip tools -> force final answer).
6. Capture the child's last assistant text; return it as the parent's `tool_result`. Discard the child's intermediate messages.

```
parent loop                          child = SAME loop, isolated context
-----------                          -----------------------------------
model -> tool_use(Task, prompt) ■■■
                          ▼
                        resolve AgentDefinition(subagent_type)
                        ctx_child = clone(ctx_parent) but:
                          messages = [user(prompt)]      # fresh window
                          tools    = filter(allTools, def) - [Task]
                          agentId  = new(); abort = new()
                          model    = def.model ?? inherit
                                ▼
                        for msg in run_loop(ctx_child, stepLimit):
                            record(msg)                  # NOT in parent window
                        final_text = last_assistant_text(child)
                                ▼
parent.tool_result(tool_use_id) = final_text   ■■■ only this crosses back
parent loop continues
```

In the wild

All five repos converge on "recurse the loop"; they differ only in what the child context is made of and how the result is harvested.

Repo	Subagent =	Child context	Result
claude-code	recurse `query()`	`createSubagentContext` + filtered tools	last text -> `tool_result`
openclaude	recurse `query()`	same + `agentStepLimit`	last text -> `tool_result`
opencode	child `Session`	`parentID` + derived perms	last text part
codex	fork child thread	filtered rollout + TOML role	inter-agent message
goose	new `Agent`	recipe + filtered extensions	extracted text

- **claude-code**: `AgentTool.call()` selects an `AgentDefinition`, builds an isolated context via `createSubagentContext` (`runAgent.ts:700`), then runs `for await (... query({ ... toolUseContext, maxTurns }))` (`runAgent.ts:748`). Tools are filtered by `resolveAgentTools` / `filterToolsForAgent` against the definition's `tools` and `disallowedTools` (`agentToolUtils.ts:122,70`). The `Agent` tool is in `ALL_AGENT_DISALLOWED_TOOLS` by default (`constants/tools.ts:41`) — so the child cannot recurse. The final-text-as-`tool_result` mapping is `mapToolResultToToolResultBlockParam` (`AgentTool.tsx:1340-1373`).
- **openclaude**: identical shape (`runAgent.ts:800`), plus an explicit `agentStepLimit`: once `stepsUsed >= maxSteps` it sets `summaryRequested`, which makes the next request send an empty tool list (`query.ts:1004`), forcing a textual wrap-up instead of more tool calls (`query.ts:1957-2003`, `query/agentStepLimit.ts:1`).
- **opencode**: the `Task` tool creates a *child Session* with `parentID: ctx.sessionID` (`task.ts:145`), runs the normal `ops.prompt(...)` loop (`task.ts:188`), and returns the last text part (`task.ts:199`). Permissions are derived per-subagent via `deriveSubagentSessionPermission`.
- **codex**: `spawn_agent` forks a child *thread* (`agent/control/spawn.rs:89`) whose rollout is filtered by `keep_forked_rollout_item` — it keeps system/developer/user messages and only `FinalAnswer` assistant messages, drops tool calls/reasoning (`spawn.rs:34-68`). A child's identity comes from a **TOML role** layered onto the config (model, provider, prompt) via `apply_role_to_config` (`agent/role.rs:56`). The subagent tool surface is `spawn_agent` / wait (`tools/handlers/multi_agents_v2/spawn.rs:27`).
- **goose**: `run_subagent_task` builds a fresh `Agent::with_config`, sets its provider/model, adds only the recipe's extensions, renders a dedicated `subagent_system.md` carrying `max_turns` + the visible tool list, calls `agent.reply(...)` (the same loop), and `extract_response_text` returns the last message (or all text) (`subagent_handler.rs:48,122,233,65`).

Divergence to note: claude-code/openclaude/goose seed a *fresh* window; codex *forks* the parent's history (filtered) so the child sees prior final answers. opencode is the outlier in modeling a subagent as a first-class persisted `Session` rather than an ephemeral context.

Scenario

Parent is asked "find every call site of `parseConfig` and tell me which ones ignore the return value." The parent doesn't want 40 file reads in its own window.

1. Parent emits `tool_use` Task `{ subagent_type: "explore", prompt: "List call sites of parseConfig; flag ones that discard the result", description: "scan" }`.
2. `AgentTool.call()` resolves the `explore` definition (read-only `tools`: Grep, Read; `model: inherit`), builds a child context with those two tools only — `Agent`, `Write`, `Bash` are filtered out (`agentToolUtils.ts:122`;

`constants/tools.ts:41`).

3. The child loop runs: greps for `parseConfig`, reads ~12 files, reasons about each. All 12 reads + intermediate reasoning live in the *child* window. After, say, 18 tool steps it nears its step limit; openclaude's path would now send an empty tool list (`query.ts:1004`) to force the summary.

4. Child's final assistant text: "9 call sites. 2 ignore the return: `init.ts:40`, `cli.ts:88`." That string is captured (`runAgent.ts:748` loop -> last text).

5. Parent receives one `tool_result` with those ~3 lines (`AgentTool.tsx:1347`), never the 12 file bodies. Parent answers the user from a near-empty delta to its context window.

Net effect: the parent spent ~200 tokens of context to get the distilled result of a ~15k-token investigation.

Lean take

- **Build exactly one thing: a `task` tool that recurses your own loop.** New `Message[]` seeded with the prompt, a tool pool filtered by an allow-list, a step cap, return the last assistant text. That is the entire feature (claude-code `runAgent.ts:748`; opencode `task.ts:188`).
- **Make the agent definition a Markdown file with YAML frontmatter** (`name`, `description`, `tools`, `model`) — parse it into your existing tool/model config. No bespoke registry, no DSL (schema: `loadAgentsDir.ts:73-98`).
- **Exclude the task tool from the child's tools.** One line; it caps recursion at one level and kills the whole class of swarm pathologies (`constants/tools.ts:41`).
- **Enforce a step/turn limit by stripping tools at the cap**, not by throwing. Forcing a final text answer always returns something usable to the parent (openclaude `query.ts:1004`).
- **Skip coordinators, swarms, peer-session orchestrators, role registries, and background/remote/worktree variants.** They are product surface, not the mechanism; codex's multi-agent-v2 and claude-code's coordinator mode are both feature-gated and inessential. Add them only when a real workload proves one level of recursion is insufficient — it rarely is.

14. MCP (Model Context Protocol)

MCP is a JSON-RPC protocol that lets an agent plug in external **tools**, **resources**, and **prompts** from servers it did not write. The agent is an MCP *host*: it spawns/connects to servers, lists their capabilities, adapts each discovered tool into its native tool interface, and merges them into the registry so the model cannot tell a remote tool from a built-in.

How it works

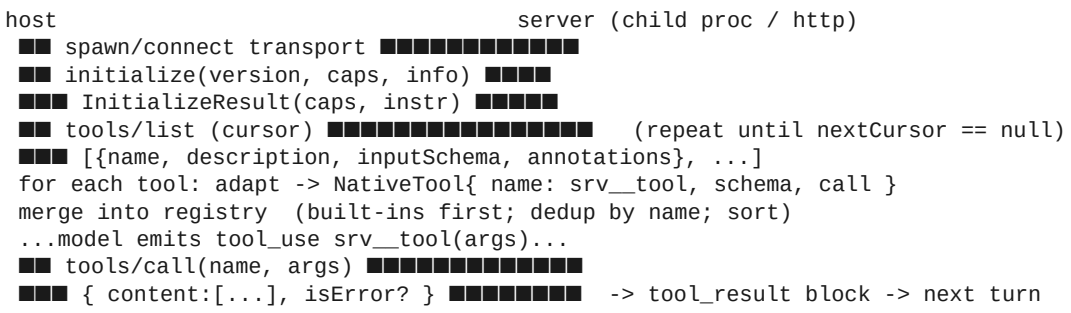
An MCP server exposes three capability families over JSON-RPC: `tools/list` + `tools/call`, `resources/list` + `resources/read`, and `prompts/list` + `prompts/get`. The host opens a **transport** to the server, performs an `initialize` handshake (exchanging protocol version + capabilities + client identity), then drives those RPCs. The host can also serve *back* to the server: `roots/list` (which dirs the agent works in), `sampling/createMessage` (server asks the agent's LLM to complete), and `elicitation/create` (server asks the user a question). Notifications flow server→host (progress, logging, `tools/list_changed`).

The single load-bearing idea: a discovered MCP tool carries `{name, description, inputSchema(JSON), annotations}` — which is *exactly* the shape of a native tool. So adaptation is mechanical: wrap the MCP tool in the host's `Tool`/`ToolExecutor` interface, pass the raw JSON schema straight to the model, and route `call()` to `client.callTool(name, args)`. The result `{content[], isError}` becomes a normal tool-result block. Built-ins win name conflicts; the merged pool is sorted for prompt-cache stability.

Transports are additive. **stdio** (spawn a child process, talk JSON-RPC over stdin/stdout) is the floor and covers most local servers. **SSE** and **Streamable-HTTP** add remote servers, with **OAuth** layered on top for auth. **In-process** (a linked transport pair, no subprocess) and **SDK/control** transports are optimizations for servers you bundle.

Flow

1. Read server config `{type, command/url, env, headers}`.
2. Build the matching transport (stdio spawns the child; HTTP/SSE attach auth).
3. `new Client(clientInfo, capabilities)`; `client.connect(transport)` runs `initialize` and stores the server's `InitializeResult` (incl. instructions).
4. `tools/list` (paginate via `nextCursor`); same for resources/prompts if needed.
5. Adapt each tool: namespace the name (`serverName__tool`), copy the JSON schema, map `annotations.readOnlyHint`→ concurrency/permission hints, bind `call()`.
6. Merge adapted tools into the registry (built-ins first, dedup by name).
7. On a tool call the loop dispatches by name; MCP tools route to `callTool`.
8. Result content → tool-result block; `isError` → error result fed back to model.



In the wild

Repo	Transports	Tool adaptation	Server mode
claude-code	5 + OAuth `client.ts:944`	raw schema in `MCPTool` `client.ts:1813`	stdio `entrypoints/mcp.ts:47`
openclaude	5 + sdk + OAuth `client.ts:652`	adapt + re-expose `mcp.ts:98`	stdio + gRPC `grpc/server.ts`
codex	stdio/HTTP/in-proc `rmcp_client.rs:47`	`ToolExecutor` `handlers/mcp.rs:67`	yes `codex_tool_runner.rs:57`
goose	5 incl. Builtin `extension.rs:163`	all are `rmcp::Tool` `mcp_client.rs:78`	host-only
opencode	stdio/SSE/HTTP+OAuth	merged, blanket-ask	—

Divergences worth noting:

- **goose makes MCP the only tool bus.** `McpClientTrait` (`mcp_client.rs:78`) is the universal interface; the built-in `developer` editor implements it in-process (`developer/mod.rs:176`) exactly like a remote server. One dispatch path, one schema format. Dispatch is `agent.rs` → `ExtensionManager::dispatch_tool_call` (`extension_manager.rs:1744`), which resolves `extension__tool` namespacing. The trait covers the *full* surface — `list_tools`/`call_tool`/`list_resources`/ `read_resource`/`list_prompts`/`get_prompt`/`subscribe` (`mcp_client.rs:103-142`) — plus host callbacks `create_message` (sampling, `:376`) and `create_elicitation` (`:463`), and forwards `tools/list_changed` to rebuild the tool list mid-loop.
- **codex wraps MCP tools in the same trait as built-ins.** `McpHandler` (`handlers/mcp.rs:67`) implements `ToolExecutor`: `spec()` exposes the server's JSON schema as a Responses-API tool (`:233`), `supports_parallel_tool_calls()` honors `read_only_hint` (`:76`), `handle()` calls `handle_mcp_tool_call` (`:145`). MCP tools are first-class registry entries, not a special path.
- **The TS repos adapt, then merge with built-ins winning.** claude-code's `connectToServer` is `memoize`d (`client.ts:595`); discovered tools are spread over a shared `MCPTool` template (`client.ts:1770`) with the **raw** server schema as `inputJSONSchema` (no Zod re-validation, `client.ts:1813`) and `checkPermissions` returning `passthrough` so the rule engine governs them (`client.ts:1814`). `assembleToolPool` sorts each partition and `uniqBy`-dedups so **built-ins win name conflicts** and stay a contiguous cache prefix (`tools.ts:345-366`).
- **Names are namespaced** to avoid collisions: claude-code `serverName__tool` (`buildMcpToolName`, `client.ts:1768`), goose `extension__tool` (`extension_manager.rs:1395`), codex `mcp__server__tool` (`handlers/mcp.rs:48,59`).
- **Annotations drive behavior, not just docs.** `readOnlyHint` → concurrency-safe
- read-only (claude-code `client.ts:1796`), parallel-eligible (codex `:76`), SmartApprove auto-allow (goose). `destructiveHint`/`openWorldHint` flow through too.
- **Resources & prompts** get surfaced as their own first-class tools in the TS repos: `ListMcpResourcesTool`/`ReadMcpResourceTool` (`client.ts:53,56`) and MCP prompts as slash-commands. goose/codex call `list_resources`/`read_resource` directly through the client trait.

Scenario

A user configures a Postgres MCP server: `{"db": {"command": "pg-mcp", "args": ["--dsn", "..."]}}`. End-to-end in claude-code:

1. At startup `connectToServer("db", ref)` sees `type: 'stdio'` (or absent) and spawns the child with `StdioClientTransport`, piping stderr away from the UI (`client.ts:944-958`).

2. `new Client(...)` declares `roots` + `elicitation` capabilities, registers a `ListRoots` handler returning `file://<cwd>`, then `client.connect(transport)` runs `initialize` under a timeout (`client.ts:985-1048`).

3. `tools/list` returns `query_sql` with `{inputSchema: {sql: string}, annotations: {readOnlyHint: true}}`. The adapter produces a tool named `mcp__db__query_sql`, `isConcurrencySafe()`→`true`, `inputJSONSchema` = the raw schema, `call()` bound to `callMCPTool` (`client.ts:1768-1840`).

4. `assembleToolPool` merges it after the sorted built-ins (`tools.ts:363`). The model now sees `mcp__db__query_sql` indistinguishable from `Read`/`Bash`.

5. Model emits `tool_use mcp__db__query_sql({sql:"select ..."})`. The permission layer sees `passthrough` and applies the user's allow rule. Dispatch calls `client.callTool("query_sql", {sql})`.

6. The server returns `{content:[{type:"text", text:"...rows..."}]}`; `transformResultContent` maps it to a tool-result block (`client.ts:2478`). Because the tool is concurrency-safe it could have batched in parallel with other reads. The result re-enters history and the loop continues.

Lean take

- **Ship stdio first, nothing else.** Spawn child, JSON-RPC over stdin/stdout, `initialize` → `tools/list` → `tools/call`. That is 90% of real-world servers and ~150 lines on top of an SDK. Add Streamable-HTTP + OAuth only when a user needs a remote server; SSE is effectively legacy.
- **Adapt, don't re-model.** A discovered MCP tool already matches your tool interface. Copy its JSON schema verbatim (do *not* round-trip through your schema lib), namespace the name (`server__tool`), and bind `call` → `client.callTool`.
- **Merge so built-ins win.** Concatenate built-ins (sorted) then MCP tools (sorted), dedup by name. Built-ins as a stable prefix keeps the prompt cache warm.
- **Honor `readOnlyHint`.** It is the one annotation worth wiring: drive parallel-eligibility and auto-approve from it for free, like codex/goose.
- **Steal goose's idea if greenfield:** make MCP your *only* tool interface and implement built-ins as in-process clients (`mcp_client.rs:78`). One dispatch path, one schema format, and the whole MCP ecosystem for free.
- **Skip until asked:** resources/prompts (surface lazily as their own tools), sampling, elicitation, `list_changed` hot-reload, and **server mode** (exposing your agent as an MCP server, ~100 lines of `ListTools`/`CallTool` handlers — claude-code `entrypoints/mcp.ts:59,99`) are all additive product surface.

15. Sessions, Persistence, Todos & Background Tasks

A **session** is a persisted conversation you can stop, resume, and fork. The kernel (loop + tools + provider) only needs an in-memory `Message[]`; everything in this chapter is the runtime/product layer that gives that array durability, a resume handle, a model-managed scratchpad (todos), and a way for long-running shells to feed results back into the loop. All of it is *additive* — you can ship a working agent with none of it.

How it works

Four loosely-coupled mechanisms, all hanging off one identifier (the session id):

- **Persistence.** Every message/part is appended to durable storage as it is produced. Two shapes dominate: an append-only **JSONL rollout** (codex, openclaude, claude-code transcripts) and a **SQLite** table of messages (opencode, goose). JSONL is trivial to append and `tail`; SQLite gives you indexed queries, monotonic sequence ids, and transactional multi-row updates (todos) for free.
- **Resume.** Read the persisted records back into the loop's history. With JSONL you parse the file into items; with SQLite you `SELECT ... ORDER BY seq`. opencode goes furthest: it never holds a mutable conversation object at all and **reloads history from SQLite every turn**, so resume is the normal code path.
- **Fork.** Copy a transcript prefix into a *new* session id so you can branch without mutating the original. The only subtlety is identity: remap message ids/parent links (openclaude) or record a `forked_from_id` and truncate the rollout at a turn boundary (codex).
- **Todos & background tasks.** A `TodoWrite` tool that overwrites a per-session list is a steering scratchpad the model maintains for itself. A background task is a detached shell/subagent whose completion **enqueues a synthetic message** that the loop drains on its next turn — the re-notification is the whole trick.

Flow

1. Session created with an id; storage opened (defer file creation until first write — codex precomputes the path and writes lazily).
2. Each turn appends new messages/parts (and a token/cost rollup) to storage.
3. `TodoWrite` calls replace the session's todo rows; the tool result nudges the model to keep using the list.
4. A tool may spawn a background job: the loop continues; the job runs detached.
5. On completion the job enqueues a `task-notification` (or writes a result part) keyed to the session; the loop picks it up as an attachment next turn.
6. Resume: reload persisted history (+ todos) into the loop. Fork: copy a prefix into a fresh id, rewriting identity.

```
# Append-as-you-go persistence + resume + fork (storage-agnostic)
on each produced message m:
   storage.append(sessionID, m)           # JSONL line OR SQLite row(seq++)

resume(sessionID):
   history = storage.load(sessionID)      # parse JSONL  /  SELECT ORDER BY seq
   return runLoop(history)                 # same kernel, pre-seeded

fork(srcID, cutPoint) -> newID:
   items = storage.load(srcID)
   prefix = items[.. cutPoint]            # nth user msg / turn id / messageId
   newID = fresh()
   for it in prefix:
       storage.append(newID, remap_identity(it, src=srcID))  # new ids/links
   record(newID).forked_from = srcID
   return newID

# Background task re-notifies the loop instead of blocking it
```

```
spawn_bg(cmd): job = detach(cmd); return job.id          # loop keeps going
on job done:  queue.enqueue({sessionID, kind:"task-notification", text})
loop turn:    history += drain(queue).map(as_attachment)  # model sees result
```

In the wild

Repo	Storage	Resume / Fork	Todos
codex	JSONL rollout, bg writer	resume reopens; fork = meta ptr + truncate	(plan tool)
opencode	SQLite (Drizzle)	reload+rederive each turn	`todo` table, txn replace
openclaude	JSONL per session	parentUuid chain; fork remaps uuids	`appState`, rehydrated
claude-code	JSONL transcript	restore from log	`appState.todos` in-memory
goose	SQLite (sqlx)	`copy_session`; truncate from msg	(per-turn flags)

- **codex rollout** is an append-only JSONL file written by a dedicated background Tokio task behind an `mpsc` channel, so the loop never blocks on disk (`codex-rs/rollout/src/recorder.rs:842`, writer `:1718`). It is created lazily (path precomputed, file opened on first `persist`, `recorder.rs:744`) and carries `forked_from_id`/`parent_thread_id` in its first `SessionMeta` line (`recorder.rs:85`). Resume reopens for append (`recorder.rs:813`); load parses each line into a `RolloutItem`, taking the first `SessionMeta` as canonical id (`recorder.rs:933`, `get_rollout_history` `:998`).
- **codex fork** is rollout truncation at a turn boundary: `truncate_rollout_after_turn_id` cuts after a persisted `TurnStarted` (`codex-rs/core/src/thread_rollout_truncation.rs:162`) or before the nth user message (`:136`); the child spawns with that history and a `forked_from_id` (`codex-rs/core/src/codex_delegate.rs:89`).
- **opencode** is the SQLite-source-of-truth design. `message`/`part`/`todo` tables (`packages/core/src/session/sql.ts:68,82,100`); history loads via `SessionHistory.load` (drop pre-compaction rows by latest compaction seq, `core/src/session/history.ts:66`); the loop **reloads every turn** and exits off rederived state, not a mutable object (`opencode/src/session/prompt.ts:1088`, exit `:1111`). Todos are a transactional delete-all-then-insert keyed by `(session_id, position)` (`core/src/session/todo.ts:32`).
- **openclaude** stores one JSONL file per session under `~/.claude/projects/<dir>/ <id>.jsonl`; messages chain by `parentUuid`. Resume walks the chain from the leaf back to root (`src/entrypoints/sdk/sessions.ts:148`). **Fork** copies the prefix (optionally `upToMessageId`), generates a new session id, and **remaps every uuid + parentUuid**, stamping `forkedFrom` on each line (`sessions.ts:325`). Mutations like rename/tag are just appended JSONL entries (`:244`).
- **claude-code** persists a transcript too; on resume it rebuilds state from the log, including **scanning the transcript for the last `TodoWrite` block** to rehydrate the todo list (`src/utils/sessionRestore.ts:77,138`). Todos themselves live in `appState.todos` keyed by `agentId ?? sessionId` (`src/tools/TodoWriteTool/TodoWriteTool.ts:65`); completing the whole list clears it, and the tool result always re-injects "continue to use the todo list" (`:104`).
- **Background tasks (claude-code)** are the clearest re-notify pattern. A detached shell, on completion, calls `enqueuePendingNotification` with a `task-notification` payload at priority `later` (`src/utils/messageQueueManager.ts:142`, emitted from `src/tasks/LocalShellTask/LocalShellTask.tsx:166`). The loop drains the queue each turn and converts those commands into **attachment messages** appended after tool results (`src/query.ts:1570,1580`).

`TaskOutputTool` lets the model pull a job's buffered stdout from disk on demand (`src/tools/TaskOutputTool/TaskOutputTool.tsx:208`).

- **Background tasks (opencode)** are a process-local registry of scoped Effect fibers with `start/wait/promote/cancel` and a `promoted` deferred (`packages/core/src/background-job.ts:120`); explicitly **not durable** (a restart loses live work, `:113`). A background subagent runs as a job and injects a synthetic result message when done (`opencode/src/session/prompt.ts:255`). Promotion of a queued input is tracked in SQLite via `admitted_seq`/`promoted_seq` (`core/src/session/sql.ts:140`, `core/src/session/input.ts:41`).

Divergences worth noting: JSONL (append, greppable, parse-to-resume) vs SQLite (indexed, monotonic seq, transactional todos); fork-by-uuid-remap (openclaude) vs fork-by-meta-pointer-and-truncate (codex); todos persisted in their own table (opencode) vs todos as ephemeral app state rehydrated from the transcript (claude-code); background jobs durable-via-DB-pointer (opencode inputs) vs purely process-local (opencode jobs, claude-code shells).

Scenario

A user runs a long task, backgrounds a test build, then resumes tomorrow (claude-code-style transcript + queue):

1. Turn 5: model calls `TodoWrite` with 4 items, item 1 `in_progress`. The tool writes `appState.todos[sessionID]`, returns "Todos modified... continue to use the todo list"; the `tool_use`+`tool_result` pair is appended to the JSONL transcript like any other.
2. Turn 6: model runs `npm run build` with `run_in_background: true`. The shell detaches; the tool returns a `taskId`; the loop **does not block** and goes on to edit files in turns 7-9.
3. The build finishes during turn 9. `LocalShellTask` enqueues a `task-notification` (`<task-notification><task-id>…</task-id><status>completed </status>…`) at priority `later`.
4. Turn 10 begins: the loop drains the queue, wraps the notification as an attachment user message, and appends it after the tool results. The model now sees the build succeeded, calls `TaskOutput` to read the log, marks todo item 4 `completed`, and (all items done) the list clears.
5. The user quits. Next day, `--resume <sessionID>` reads the transcript back into `Message[]`; `restoreSessionStateFromLog` scans backward for the last `TodoWrite` block and rehydrates `appState.todos`. The loop continues with full context and an accurate task list — no live state survived the restart, only the transcript did.

Lean take

- **Persist append-only JSONL, one file per session.** One line per message/part, plus a header line with `{id, cwd, createdAt, forkedFrom?}`. Resume = read + parse. It is the lowest-effort durable format and you can `tail` and `grep` it. Reach for SQLite only when you need indexed listing or transactional todo rows.
- **Make resume the normal path, not a special case.** If you can afford it, copy opencode: reload history from storage at the top of each turn so cancel/resume/ compaction are correct by construction. Otherwise, load-once-then-append is fine — just append *as you go*, never "save on exit" (crashes lose everything).
- **Fork = copy a prefix into a new id.** Cut at a user-message/turn boundary, give the new session a fresh id, and either remap message ids (if they're referenced) or just record `forked_from`. Don't deep-copy side state (file undo history, snapshots); start the fork clean.
- **Todos are one tool that overwrites a list + a nudge.** Schema is `[{content, status, activeForm}]`; the call replaces the whole list; the tool result says "keep using it." That single feedback string is what actually steers long tasks. Persisting todos is optional — rehydrating from the last `TodoWrite` in the transcript works fine.
- **Background tasks: re-notify, don't block.** Spawn detached, return an id immediately, and on completion **enqueue a synthetic message** the loop picks up next turn; expose a `TaskOutput`-style tool to pull buffered

output on demand. Keep the registry process-local at first — durable background work (survive restart, remote workers) is a large separate effort; do not pretend an in-memory map has those semantics.

- **Skip for v1:** session search/pagination, archived/compressed rollouts, state-DB-plus-filesystem reconciliation (codex), input admit/promote sequencing, worktree/remote background sessions, and per-day transcript rotation.

16. End-to-End Scenarios

The previous chapters dissected components in isolation. This one runs them together. Each scenario is a trace: the inputs, how the message list and control flow evolve, and which mechanism is doing the work. Notation: `messages` is the ordered conversation; `A` = assistant, `U` = user/tool-results.

Scenario 1 — A coding task, from prompt to stop

The spine of every session. User: *"make the failing test in `sum.ts` pass."*

```
messages:
[0] U  task + <system-reminder: CLAUDE.md memory>  + <skill listing>
-- turn 1 --
[1] A  text("Let me look at the test.") + tool_use(read, {path:"sum.test.ts"})
[2] U  tool_result(<test source>)                  # read is read-only: no prompt
-- turn 2 --
[3] A  tool_use(read,{path:"sum.ts"})              # batched? no, sequential here
[4] U  tool_result(<impl source>)
-- turn 3 --
[5] A  text("The bug is the operator.") + tool_use(write,{path:"sum.ts",...})
       --> write needs approval -> gate asks user -> "always" -> rule appended
[6] U  tool_result("Wrote 412 bytes to sum.ts")
-- turn 4 --
[7] A  tool_use(bash,{command:"npm test"})         # gate: matches allow rule
[8] U  tool_result("Tests: 1 passed")
-- turn 5 --
[9] A  text("Fixed: the operator was inverted. Tests pass.")   # NO tool_use
       --> exit test fires: no tool_use blocks => loop returns
```

What fired, in order: prompt construction injected memory + the skill listing (Ch. 5, 9, 10); each turn streamed text then a tool call, the JSON assembled at block close (Ch. 4); read ran unprompted while write went through the allow/ask/deny gate, where "always" appended a runtime rule (Ch. 7); every `tool_use` got a paired `tool_result` (Ch. 2); the loop stopped on the **absence of a tool call**, not `stop_reason` (Ch. 1).

Scenario 2 — A long session crosses the compaction threshold

Turn 30 of a refactor. Before the model call, the context manager estimates tokens.

Trace

1. Top of the loop: `estimateTokens(messages) = 156k`, window 200k, threshold `0.8 * 200k = 160k`. Under — proceed. Three turns later: `163k > 160k`. Trigger.
2. Run a **separate** summarization model call over the transcript → an 8-section summary (intent, files touched, decisions, current work, next step).
3. Rebuild: `messages = [initialContext] + [recentUserMessages <= ~20k] + [summary]`. Old turns are marked **agent-invisible**, not deleted (Ch. 6).
4. Crucially, the rebuild preserves the **pairing invariant** — no orphaned `tool_use` is left without its `tool_result`, or the next call would 400 (Ch. 2).
5. The loop continues with a ~40k-token context. The user sees no interruption; the full transcript is still on disk for replay.

```
before:  [init][t1][t2]...[t29][t30]                ~163k tokens (agent-visible)
compact: summarize(all) -> S
after:   [init][t28][t29][t30][S]                   ~40k tokens
       (t1..t27 still stored, flagged agent_invisible)
```

Reactive variant: if the estimate is wrong and the provider returns a real context-length error mid-loop, the same compaction runs as a recovery branch, then the turn retries (Ch. 6). Same machinery, different trigger.

Scenario 3 — A hook blocks a dangerous edit

The user has a `PreToolUse` hook forbidding edits to `.env` (Ch. 8).

```
 [k]   A  tool_use(Edit, {file_path:".env", ...})
       --> PreToolUse fires BEFORE the permission gate:
           build hook input JSON -> spawn matching commands (stdin=JSON)
           hook: jq sees ".env" -> prints reason to stderr -> exit 2
           aggregate: any block wins -> behavior = deny
[k+1] U  tool_result(is_error: "Edit hook error: <reason>")   # Edit NEVER ran
[k+2] A  text("That file is protected; I'll skip it and continue with config.ts.")
```

The denial is a normal `is_error` tool result — the model reads it and routes around it; the loop never throws (Ch. 1, "everything is a tool result"). Had the hook printed `{"hookSpecificOutput":{"permissionDecision":"allow","updatedInput": {...}}}` instead, the executor would have rewritten the args and let a *sanitized* edit through. Hooks sit upstream of the gate, so they can veto even otherwise-allowed actions.

Scenario 4 — A skill activates (progressive disclosure)

`.claude/skills/pdf/SKILL.md` exists; its 3K-token body is in memory but not in context (Ch. 10).

Trace

1. Each turn, the runtime injects only new skill *descriptions* as a system-reminder: `- pdf: Fill and extract PDF forms` (~12 tokens). The body is withheld.
2. User: *"fill `form.pdf` with my address."* The model matches the listing and emits `tool_use(Skill, {skill:"pdf", args:"form.pdf"})`.
3. Gate: `pdf` has only safe frontmatter → auto-allow. The Skill tool reads `SKILL.md`, prepends `Base directory: <dir>`, substitutes `$ARGUMENTS=form.pdf`, and returns the expanded body as a **new user message**; `allowed-tools` scopes the turn to `Bash/Read/Write`.
4. Next turn the model has the full workflow and runs `bash python ${CLAUDE_SKILL_DIR}/scripts/fill.py form.pdf`.

The 3K-token instruction set entered context **only for this task** — a hundred idle skills would still cost only their one-line descriptions.

Scenario 5 — Delegating to a subagent

The model decides a scoped search is best run in isolation (Ch. 13).

```
 [m]   A  tool_use(Task, {description:"find all callers of deprecated foo()",
                          subagent_type:"Explore"})
       --> the Task tool RECURSES the same loop with a CHILD context:
           child.messages = [ U(description) ]
           child.tools    = filtered set (read-only for Explore)
           child.budget   = its own token/step limit
           child runs its own turns ... returns final text
[m+1] U  tool_result(<child's final summary of callers>)     # parent's result
[m+2] A  ... parent continues with the summary, never seeing the child's turns
```

There is no separate "agent runtime": a subagent is the loop, recursed, with a narrower tool set and its own message list (Ch. 1, law 7). The parent context stays clean — it receives only the child's conclusion, not its intermediate tool calls. Swarms and coordinators (omitted in Ch. 17) are elaborations of this one move.

Scenario 6 — Calling an MCP tool

At startup the agent connected to an MCP server (e.g. a database tool) over stdio (Ch. 14).

Trace

1. Startup: spawn the server, `initialize`, `tools/list` → adapt each remote tool into the native interface and **merge into the registry**. To the loop, the tool `db.query` is indistinguishable from a built-in.
2. The merged tool's JSON schema goes to the model alongside built-ins (Ch. 3).
3. Model emits `tool_use(db.query, {sql:"select count(*) from users"})`.
4. The registry routes the call to the MCP client, which sends `tools/call` over the transport and returns the result as a normal `tool_result`.
5. The model continues. MCP tools pass through the same permission gate; many runtimes apply a blanket "ask" to external tools by default.

The only added machinery is an adapter at registration time. Everything downstream — schema rendering, the gate, pairing, results — is the same path as a built-in.

Scenario 7 — A streamed tool call, with a retry

What actually happens on the wire during one model call (Ch. 4).

```
POST /v1/messages (stream:true)  ->  529 overloaded
 withRetry: backoff = BASE * 2^(n-1), capped; honor retry-after; attempt 2
SSE events:
 message_start
 content_block_start {type:text}            -> open text block 0
 content_block_delta {text:"Reading "}      -> append
 content_block_delta {text:"the file."}     -> append
 content_block_stop                          -> close block 0
 content_block_start {type:tool_use, name:"read", id:"tu_1"}
 content_block_delta {input_json:"{\"pa"}   -> buffer fragment (NOT parsed)
 content_block_delta {input_json:"th\":\"a.ts\"}"} -> buffer fragment
 content_block_stop                          -> NOW JSON.parse the buffer -> {path}
 message_delta {stop_reason:"tool_use"}      -> recorded, but NOT the exit signal
 message_stop
=> assistant content = [ text, tool_use(read,{path:"a.ts"}) ]
=> loop collects 1 tool_use -> executes -> continues
```

Two details carry the design: tool-call arguments are **accumulated across deltas and parsed only at `content_block_stop`** (parsing a half-streamed fragment would throw), and `stop_reason` is *recorded but not trusted* — the loop's exit decision reads the assembled content for tool-use blocks (Ch. 1, 4). The retry wraps the whole call, so transient `529`s never reach the loop as errors.

What these scenarios share

Every flow above is the same five moves: assemble the request, stream a reply, look for tool calls, run them through the gate (with hooks upstream), feed paired results back — and stop only when no tool call is requested. Skills, subagents, MCP, and compaction do not change the loop; they change *what a turn contains* or *what a tool does*. That invariance is the whole point: build the loop once, correctly, and every feature in this book attaches to it without rewriting it.

17. The Lean Build

You have seen every component. This closing chapter is the opinionated assembly order: the minimal kernel, what to deliberately omit, and a staged path from a 50-line toy to a real agent. It condenses `docs/BLUEPRINT.md`; every recommendation traces to a repo studied in this book.

The minimal kernel

Six components do the real work. The last two are deferrable.

Component	Contract	Chapter
Agent loop	turn loop; exit on no `tool_use`	1
Message list	ordered `Message[]`, pairing invariant	2
Tool + registry	`{name, schema, call, isReadOnly}` + a flat list	3
Provider client	`stream(system, msgs, tools) -> events`	4
Context manager	token estimate + compaction at threshold	6
Approval gate	`decide(tool, input) -> allow/ask/deny`	7
MCP client	adapt external tools into the registry (later)	14
Subagents	recurse the loop with a child context (later)	13

Everything in this book outside that table is optional. Build the six, keep one provider seam, and add nothing speculatively.

What to omit (and why)

These are real features in the five repos that a lean framework should **not** build up front. Each is a product or scale concern, not part of the kernel.

Product / personality

- Companion/Tamagotchi, proactive "keep-going" nudges, voice/STT, ads (openclaude, goose) — persona and input-modality, irrelevant to doing work; nudges actively fight the clean "stop on no tools" rule.

Telemetry / business

- Analytics, OTel, feature-flag gating, cost tracking, LLM session-titling (all repos) — observability is additive; don't thread it through the loop on day one.

UI / surfaces

- The entire TUI (claude-code's `main.tsx` is hundreds of KB), desktop/Electron, web console, IDE bridges, and ~25–40 per-tool React renderers — your loop should emit *events*; a UI consumes them separately. Keep tools to `{schema, call}`.

Provider / context over-engineering

- A 12-provider zoo, gateway registries, dual SDK+hand-rolled parsers, 5-layer compaction, server cache-editing, WebSocket transport with HTTP fallback — ship one provider and one good summary pass; add a translator seam only for provider #2.

Permissions / safety over-engineering

- Four permission modes, LLM security/egress classifiers, repetition inspectors, three OS sandbox backends, a network MITM proxy — start with one ordered rule list
- read-only auto-allow + a bypass-immune safety floor; sandboxing can come later.

Orchestration / cloud / enterprise

- Coordinator swarms, peer-session orchestrators, role registries, gRPC/MCP server modes, remote/background sessions, SQLite persistence, snapshot/revert, an Effect/DI runtime, a plugin hook bus — one level of "subagent = recurse the loop" is the 95% case; plain async + an in-memory `Message[]` is enough to be correct.

Self-extension: the harness that rewrites itself

Omitting features is only safe if they are cheap to add back. The pi.dev insight is to make the harness **extensible by the agent itself**: expose tools (and commands) as small code modules in an `extensions/` directory, load them at startup, and give the agent a `reload_extensions` tool. Because the agent already has `write` + `edit`, it can author a new tool on demand and load it mid-session — *"if a capability does not exist, you ask the agent to build it."* This is the whole of "self-building": it is not self-improving AI, just code-writing plus hot-reload.

Flow

1. At startup, scan `extensions/`, import each module, let it register tools onto the registry (which is already just a list, Ch. 3).
2. Rebuild the tool array **every turn**, so a newly-registered tool shows up in the next model call.
3. Expose `reload_extensions` as a built-in. When the agent writes a new extension file, it calls reload and the tool becomes usable immediately.

```
# an extension is a module that adds tools; the agent can write one itself
register(pi):
   pi.addTool({ name, description, inputSchema, run })

loadExtensions(dir):
   for file in dir/*.ts:
       mod = import(file + "?v=" + now)      # cache-bust the URL => hot reload
       mod.register(api)                      # api.addTool(...) collects tools
   return tools

# in the loop, rebuilt per turn:
tools = [ ...builtins, reloadTool, ...loadExtensions(dir) ]
```

The caveat pi makes explicit: it ships **no permission system** and trusts a sandbox/container instead. If an agent can write and load its own tools, keep the approval gate (Ch. 7) or run the whole thing sandboxed — self-extension

widens the blast radius of a bad instruction. The reference implementation in `prototype/` takes the first option: `src/extensions.ts` is the loader, `reload_extensions` is the built-in, and an extension tool goes through the same gate as a built-in, with `buildTool`'s fail-closed default treating a tool that forgets to declare `readOnly` as a writer. Watch it happen, offline:

```
cd prototype && node agent.ts --mock --yes "build a shout extension and use it"
```

Staged build order

Each stage is independently runnable and testable. Do not skip ahead.

v0 — Loop + 3 tools (no streaming, no approval)

- The `while(true)` loop with the **no-`tool_use` exit test** (Ch. 1).
- `Message[]` in memory; the pairing invariant with back-fill on failure (Ch. 2).
- Tool interface with fail-closed defaults; implement `read`, `write`, `bash` (Ch. 3).
- One non-streaming provider call behind an injectable seam so it is testable without the network (Ch. 4).
- System prompt = identity + tool list + cwd; memory file injected as a leading user message (Ch. 5, 9).
- Done when: the agent can read a file, edit it, run a command, and stop.

v1 — Streaming + approval gate

- Streaming client: SSE parse, accumulate blocks by index, assemble tool-call JSON at block close, emit an internal event union; retry with backoff (Ch. 4).
- Approval gate: ordered last-match-wins rules, default ask, "always" appends a rule, deny -> `is_error` result; auto-allow read-only tools; safety floor for `.git`/config (Ch. 7).
- Parallelize consecutive read-only tools; serialize writers (Ch. 1, 3).
- Done when: output streams, risky writes prompt, reads run concurrently.

v2 — Context compaction

- Token estimate (byte heuristic) + threshold check at the top of the loop; one summarization pass -> rebuild `[init]+[recent]+[summary]`; mark old turns invisible; tool-result cap + disk spill; in-loop recovery on a real context-length error (Ch. 6).
- Done when: a long session compacts automatically and keeps going, and the full transcript is still recoverable.

v3 — Extensibility

- MCP client (stdio first): adapt remote tools into your registry, indistinguishable from built-ins (Ch. 14).
- Subagents: recurse the same loop with a child context, filtered tools, and a step limit; return the child's final text as the parent's tool result (Ch. 13).
- Then, only as the product demands: skills (Ch. 10), custom commands (Ch. 11), hooks (Ch. 8), and a persistence interface for sessions (Ch. 15).
- Done when: the agent can call an external MCP server and delegate a scoped subtask — with zero new machinery in the loop.

The reference implementation

Everything above is buildable in one sitting, and `prototype/` in this repo is that build: ~4,500 lines of dependency-free TypeScript, no framework, running on plain Node. It is not pseudocode — it streams from Anthropic, OpenAI, OpenRouter, or any OpenAI-compatible endpoint, and its 69-test suite runs with no API key by pointing the same binary at a local server that speaks both wire formats.

Each module is one chapter, so the book and the code can be read against each other:

Chapter	File	What it is
1, 13	`src/loop.ts`	the loop, tool execution, subagents
2	`src/types.ts`	canonical blocks, dual visibility, pairing back-fill
3	`src/tools.ts`	tool interface, ten built-ins, cap-and-spill
4	`src/providers.ts`, `src/config.ts`, `src/mock.ts`	two SSE clients, one event union, provider resolution
5	`src/prompt.ts`	cacheable prefix + dynamic suffix, reminders
6	`src/context.ts`	estimate, threshold, summarize-and-rebuild
7	`src/permissions.ts`	rule list, modes, safety floor
9	`src/memory.ts`	`AGENTS.md` discovery, tiers, `@includes`
11	`src/commands.ts`	client + markdown prompt commands
15	`src/session.ts`	JSONL persistence, resume, fork
17	`src/extensions.ts`	the self-extension loader

Four chapters are implemented **nowhere**, on purpose, and their absence is the argument of this one: MCP (Ch. 14), hooks (Ch. 8), skills (Ch. 10), and output styles (Ch. 12). Each has an obvious seam waiting for it — MCP tools merge through `mergeTools`, hooks wrap the single `runOne` call site, a UI consumes `TurnHooks` instead of `ui.ts` — which is the point: a kernel with clean seams lets you defer, and deferring is cheap only when the seam already exists.

Three things that emerged from actually running it, which pseudocode hides:

- **Compaction needs a floor, not just a ceiling.** On a small context window the system prompt and tool schemas alone can cross the threshold, so a naive proactive check compacts an empty transcript, burns a model call, makes the request *larger*, and does it again next turn. Gate the check on having something to summarize.
- **A non-interactive run must refuse, not block.** `--print` has nobody to answer a prompt, so an `ask` that awaits stdin hangs forever. Non-interactive mode has to turn `ask` into a denial the model can read and route around.
- **The exit test has a twin in the tool loop.** "One result per call, in the order asked" is not free once calls run in parallel batches: collect into a map keyed by `tool_use_id` and re-emit in the original order.

The disagreements you must resolve

The repos do **not** agree on everything; pick a side deliberately:

- **History:** resend full history each turn (codex) vs. slide a window after a compaction boundary (claude-code). Lean default: in-memory window.
- **Loop state:** mutable object reassigned per turn (claude-code) vs. re-derive from a persisted store each turn (opencode). Lean default: mutable in-memory.
- **Permissions:** fixed precedence (claude-code) vs. ordered last-match-wins rules (opencode). Lean default: last-match-wins — simpler and more powerful.
- **Sandbox:** real OS sandbox (codex) vs. ask-before-side-effect only (opencode, goose). Lean default: ask-only to start; add a sandbox when you run untrusted code.
- **Streaming:** hand-rolled SSE (codex, goose, claude-code) vs. a vendor SDK (opencode). Lean default: hand-roll one provider so you understand it.

The one point of near-unanimous agreement, and the most important line of code in the whole system: **exit the loop when the assistant emits no tool calls — never trust `stop_reason`.**